BETRIVALS

BLG 317E
TERM PROJECT

TEAM MEMBERS

Osman Yahya Akıncı

- SHOTS PAGE
- USERS

Talha Müderrisoğlu

- PLAYER PAGE INTEGRATION WITH TABLES PLAYERS AND FUT23

Bilge Bostanbaşı

- SEASON PAGE
- TEAMS PAGE

Abdullah Akcan

- MATCHES PAGE

OVERVIEW

- BetRivals is a web-based football analytics platform that integrates multiple datasets into a unified relational database. It enables users to explore and analyze performance metrics through an interactive, data-driven interface.

MOTIVATION

Football performance goes beyond final scores; advanced metrics provide deeper tactical insight.

OVERVIEW

GOAL!!!

- Build a comprehensive relational database linking all football entities.
- Provide interactive visualization and filtering.
- Enable complex analytical queries (joins, aggregations, subqueries) for richer insights.

DATASET DESCRIPTION

TABLES:

SHOT_DATA – EVENT-LEVEL SHOOTING INFORMATION WITH XG VALUES

24400 ITEMS

PLAYER / FUT23 – PLAYER ATTRIBUTES AND FIFA-BASED RATINGS

219 ITEMS

kaggle

TEAM – TEAM IDENTIFIERS AND NAMES

24 ITEMS

SEASON – SEASONAL PERFORMANCE METRICS (WINS, DRAWS, LOSSES, XG, XGA, PPDA, ETC.)

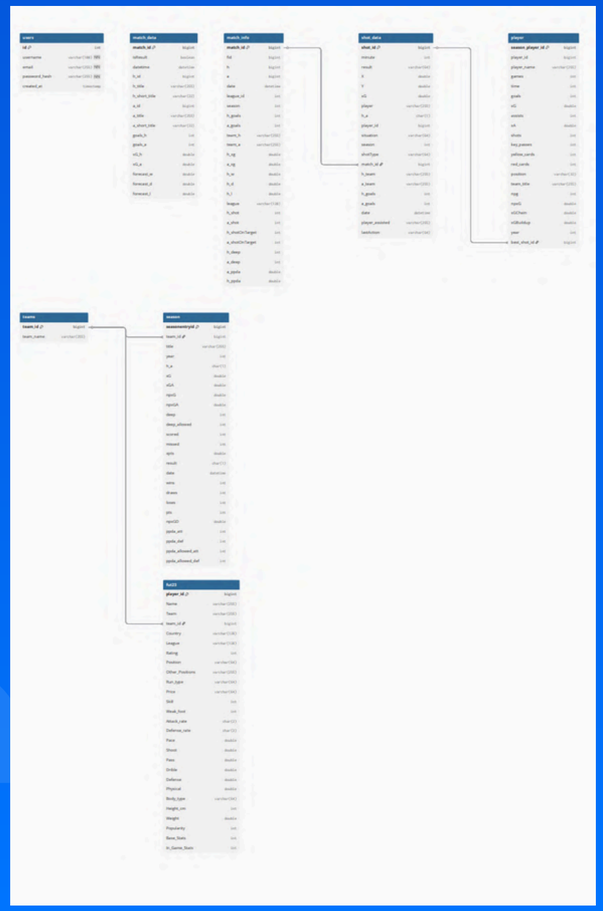
6843 ITEMS

MATCH_INFO – MATCH-LEVEL STATISTICS AND RESULTS

2954 ITEMS

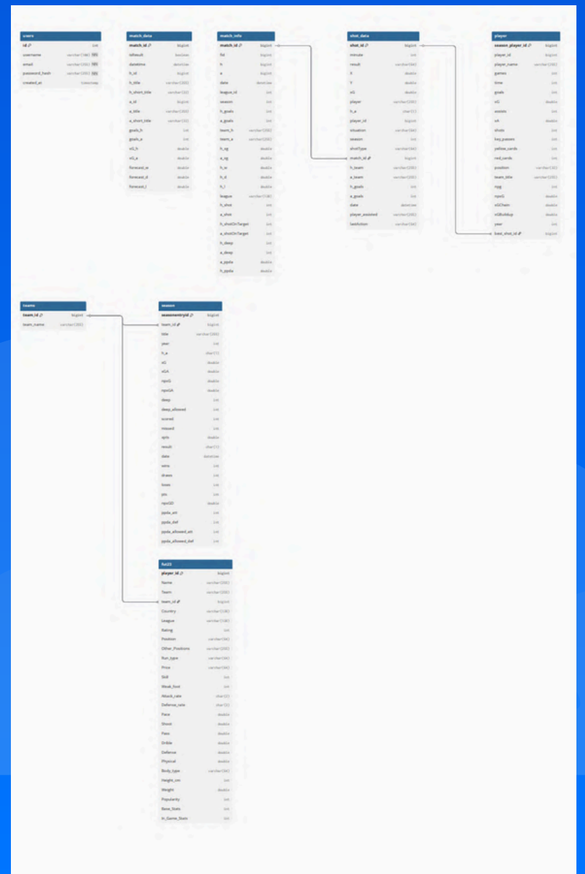
NORMALIZATION

Initial



NORMALIZATION

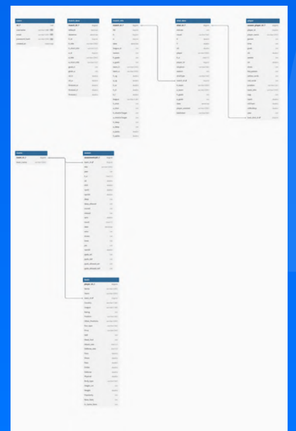
Initial



- Problem : Columns are not atomic!

NORMALIZATION

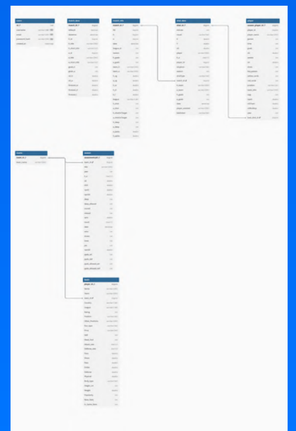
Initial



1NF

NORMALIZATION

Initial



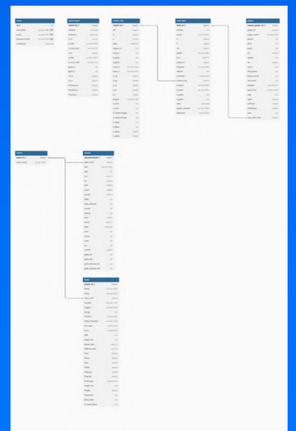
1NF

- Example Problem: “fut23" intended to track players across multiple seasons, so the true candidate key should be (player_id, year), not just player_id (Partial Dependencies Found)
- Solution: Split the fut23 table into two tables.

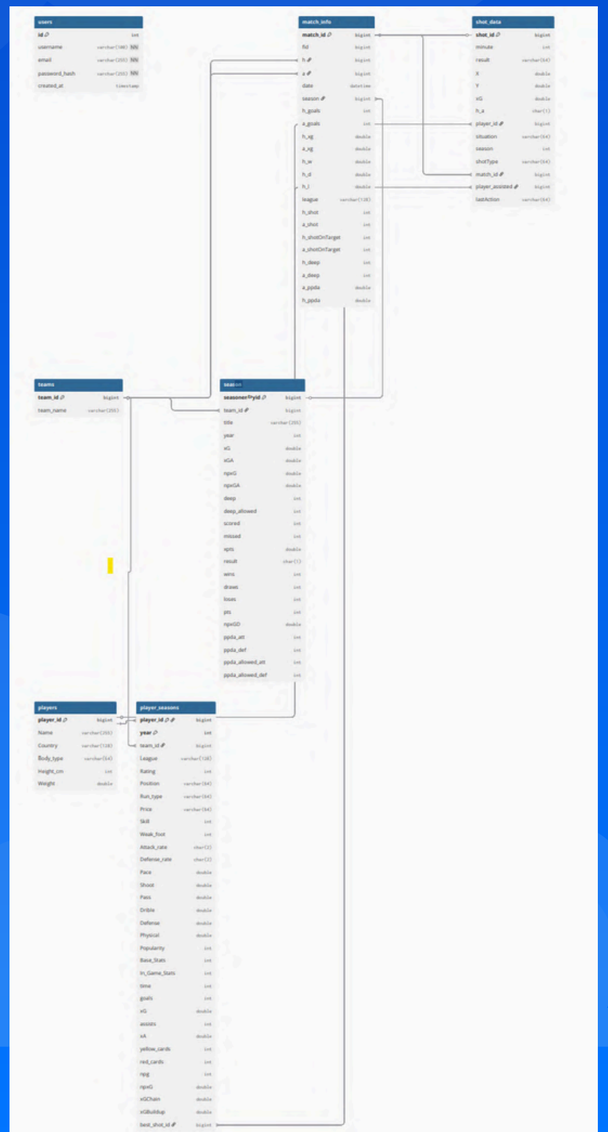
NORMALIZATION

1NF

Initial



2NF



NORMALIZATION

1NF

Initial

2NF

- There exists transitive depedencies.
- Example Solution: Removed seasos.pts because it can be calculated from wins and draws.(SELECT (wins * 3 + draws) AS pts)

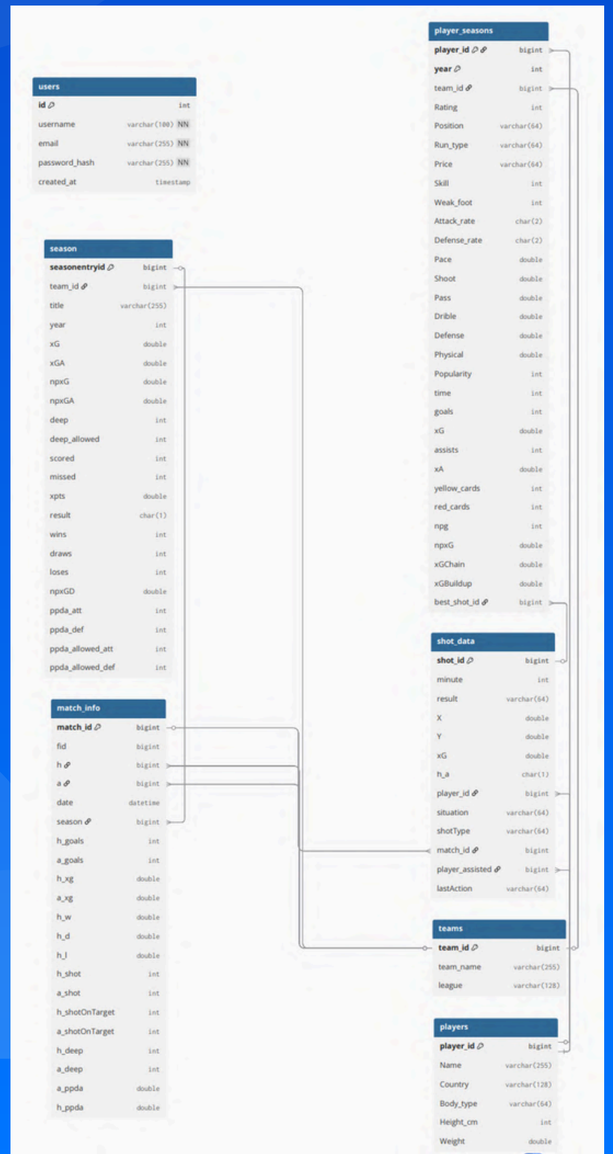
NORMALIZATION

1NF

Initial

2NF

3NF

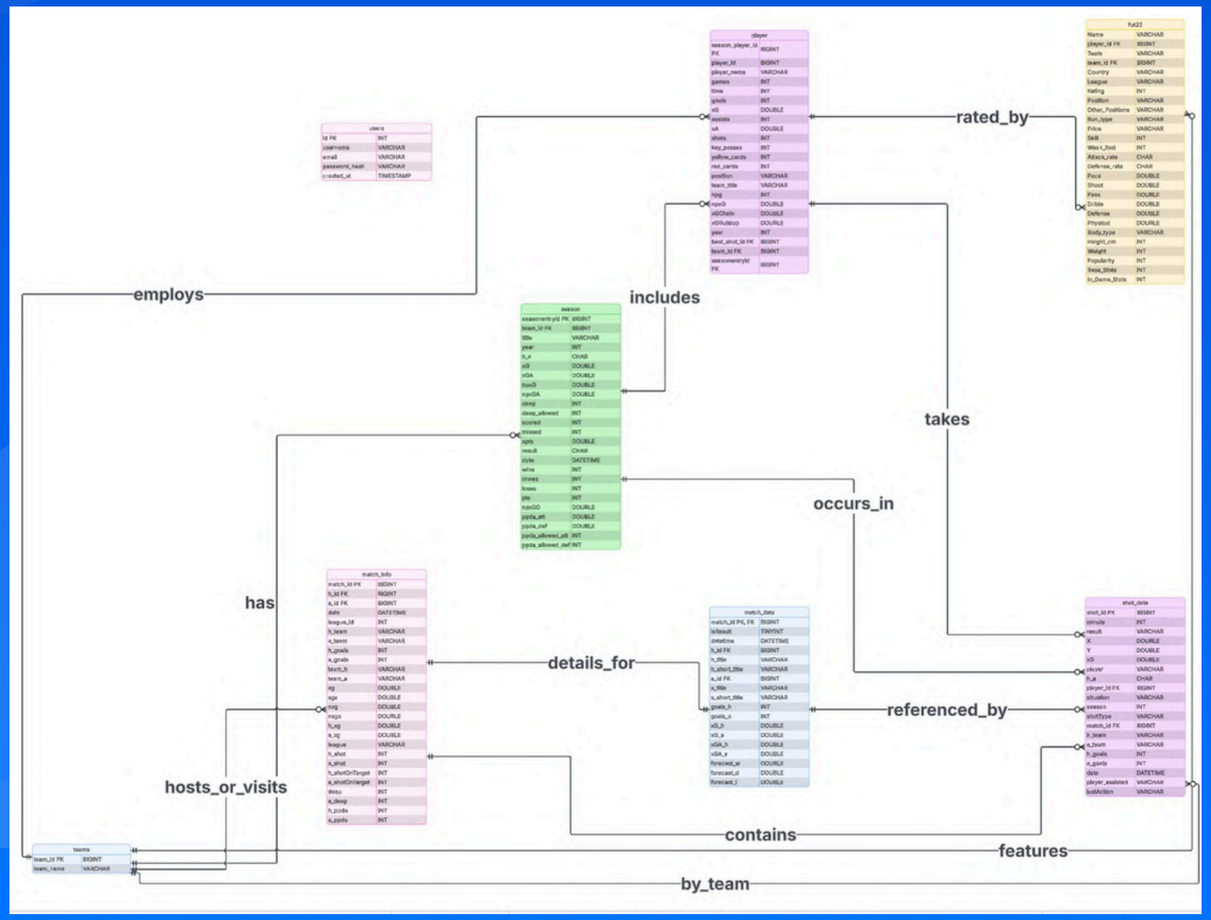


3NF is optimized enough for out project.

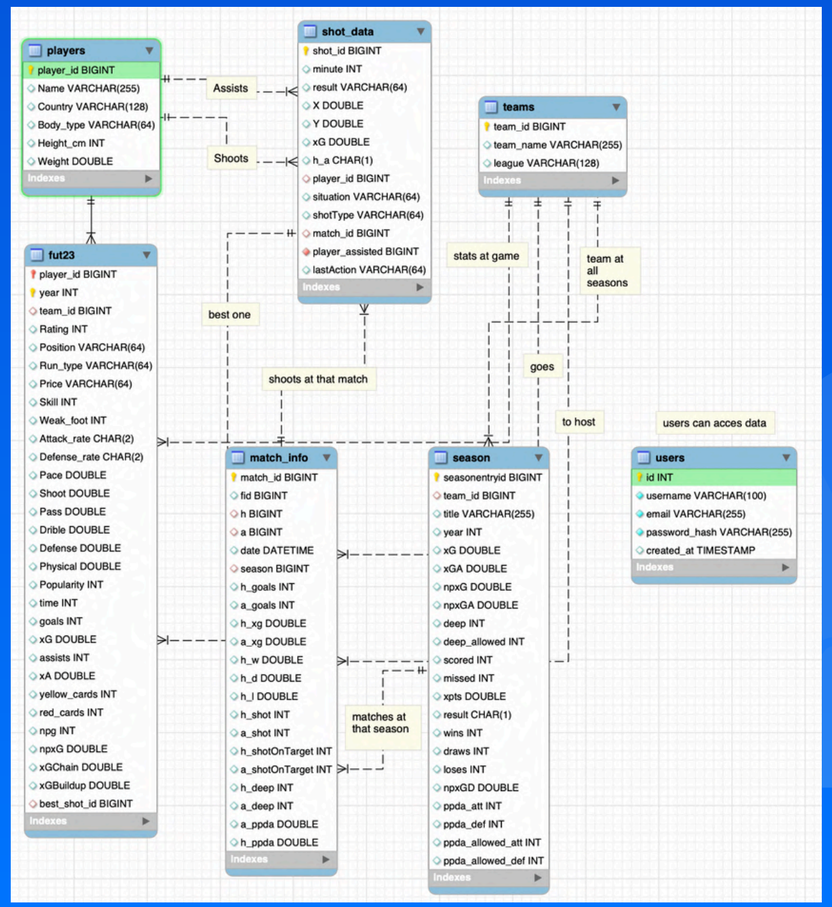
GOAL

ER-DIAGRAM

Previous

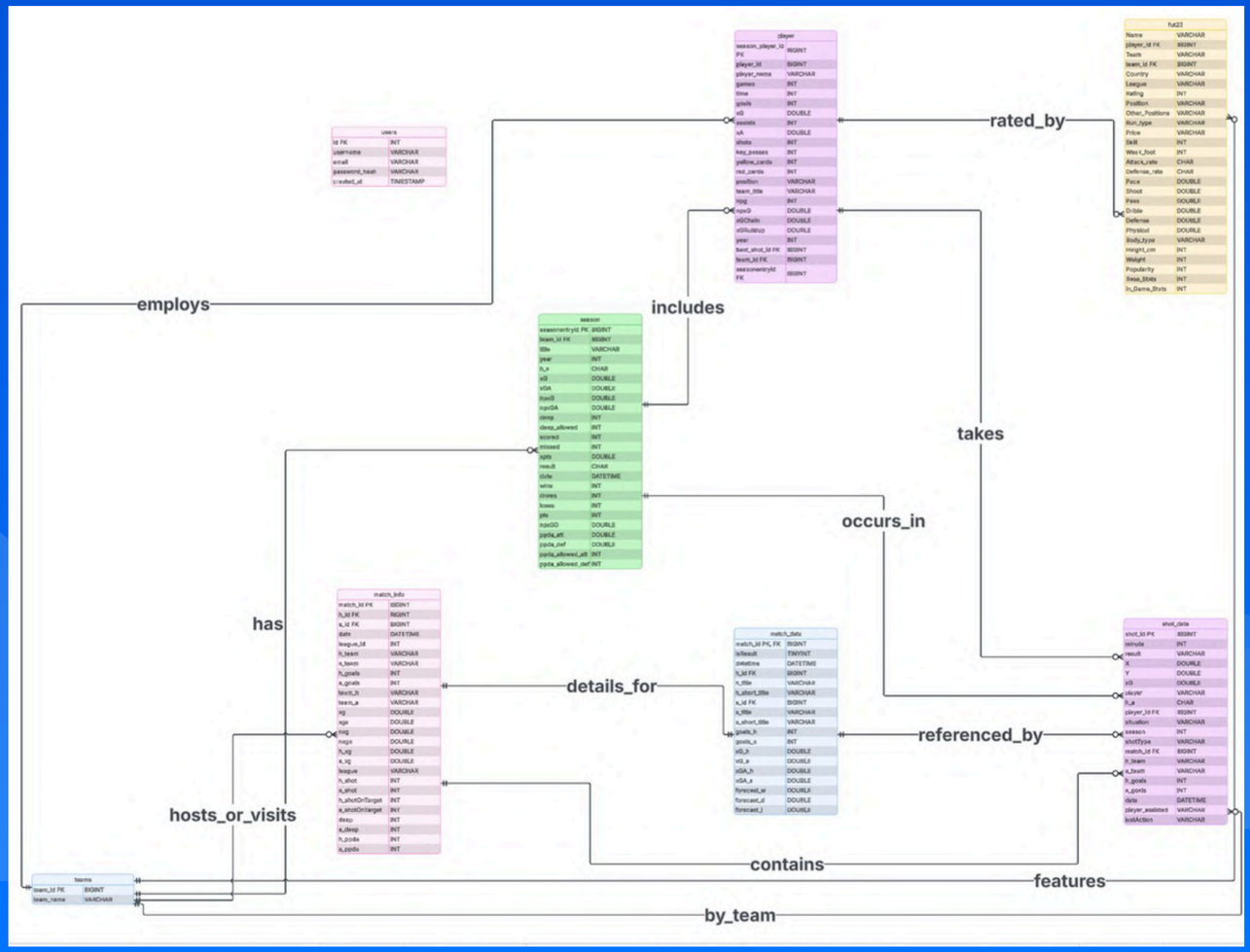


New

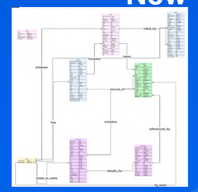


ER-DIAGRAM

Previous

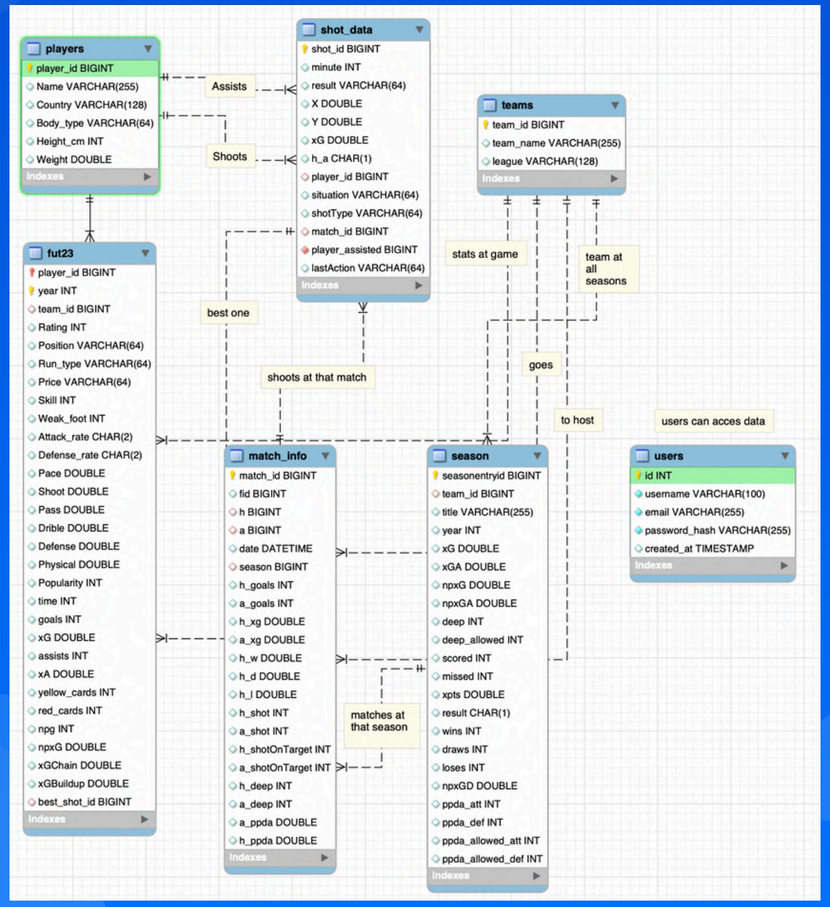


New

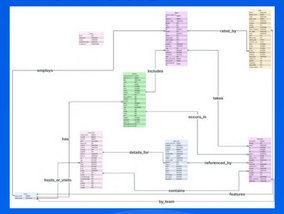


ER-DIAGRAM

New

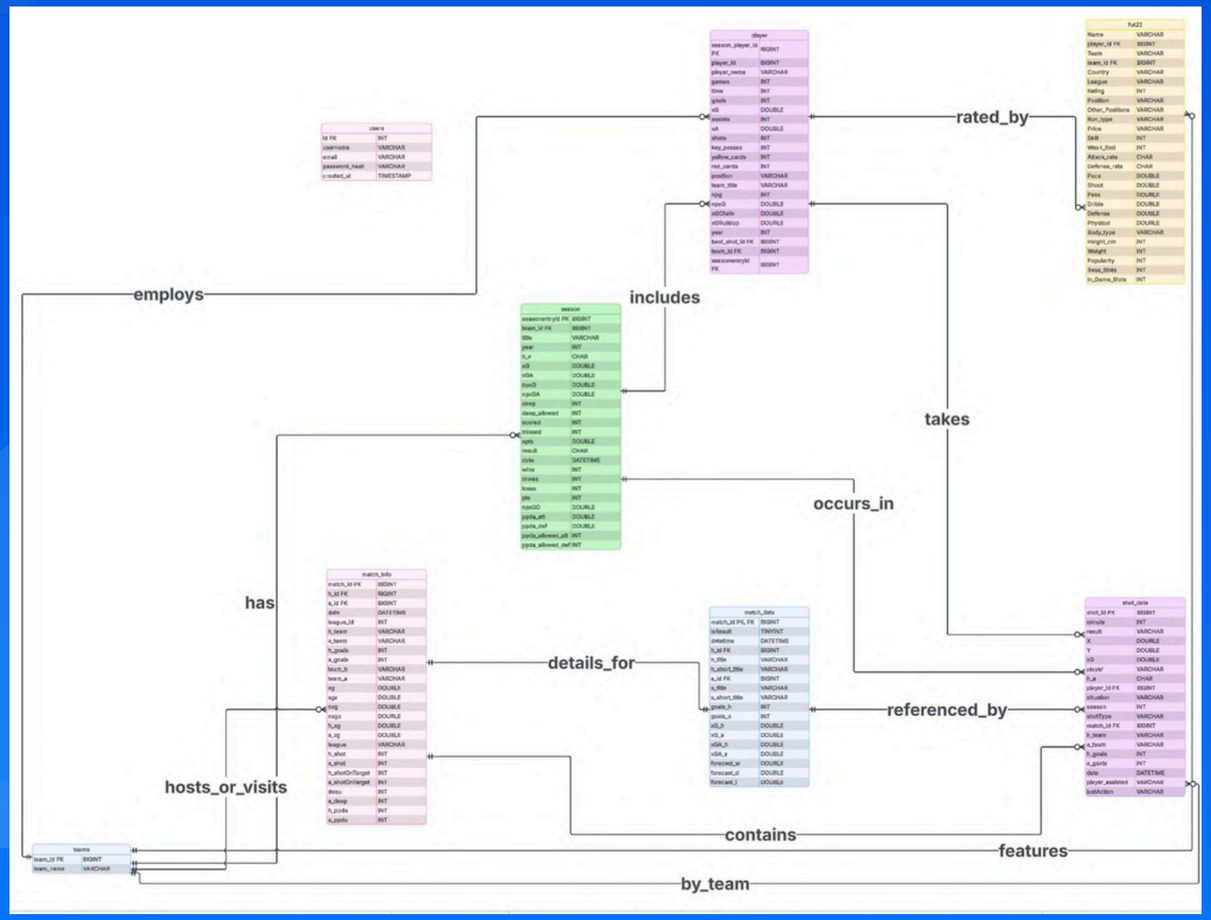


Previous

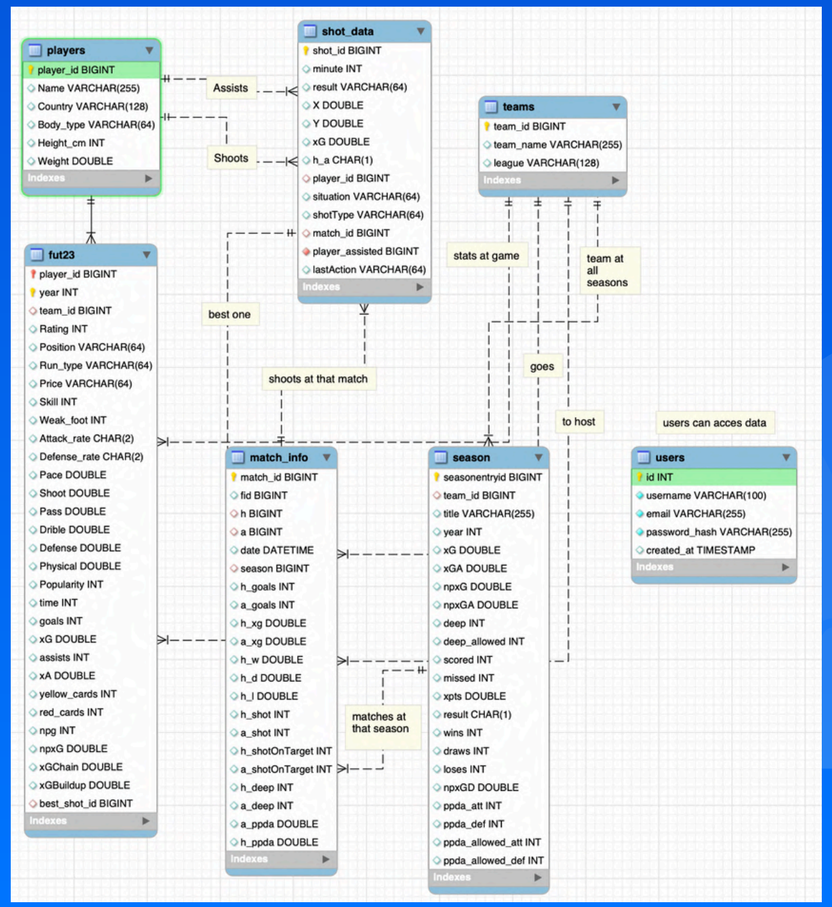


ER-DIAGRAM

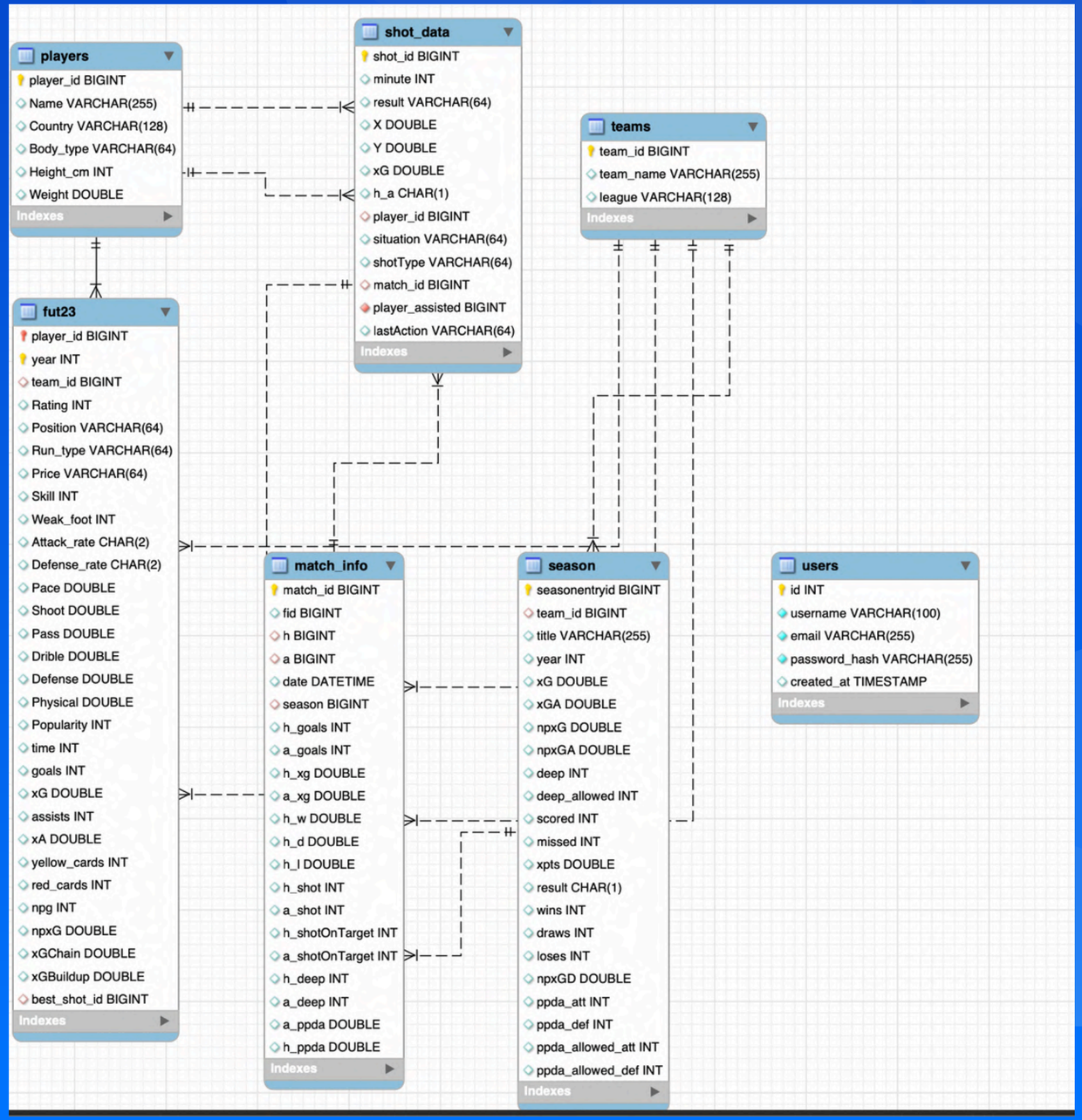
Previous



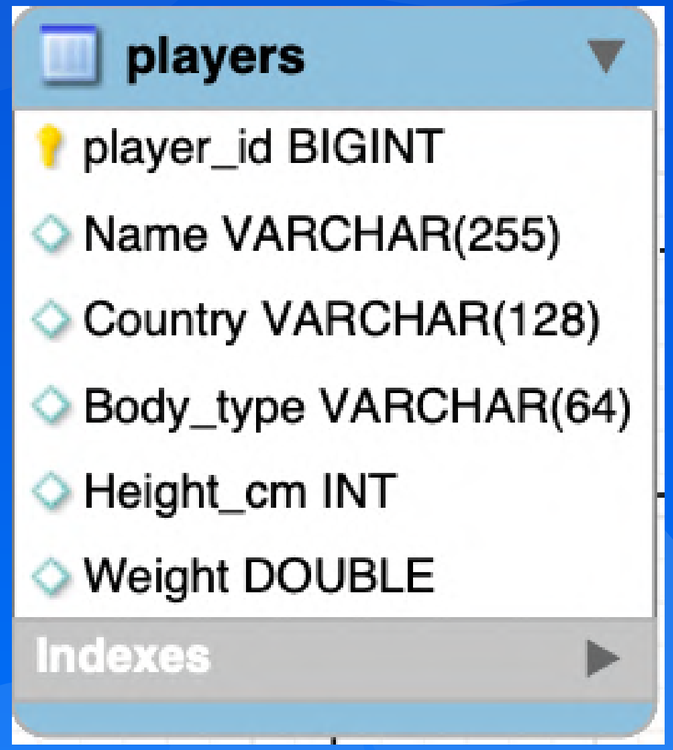
New



ER-TO-RELATIONAL MAPPING



ER-TO-RELATIONAL MAPPING

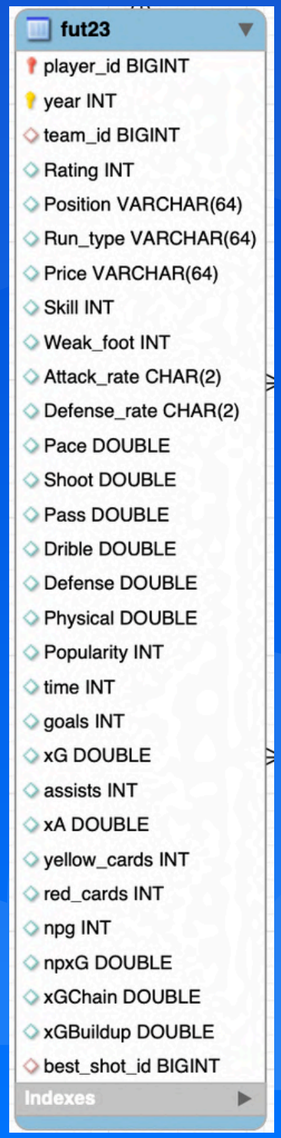


players

- Holds players data with no changing data between seasons.
- Connects to shot_data and fut 23 via PK.

```sql
-- Table 1: Player master data (attributes that don't change across seasons)
CREATE TABLE players (
    player_id BIGINT PRIMARY KEY,
    Name VARCHAR(255),
    Country VARCHAR(128),
    Body_type VARCHAR(64),
    Height_cm INT,
    Weight DOUBLE,
    CONSTRAINT uq_players_name UNIQUE (Name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

ER-TO-RELATIONAL MAPPING



fut23

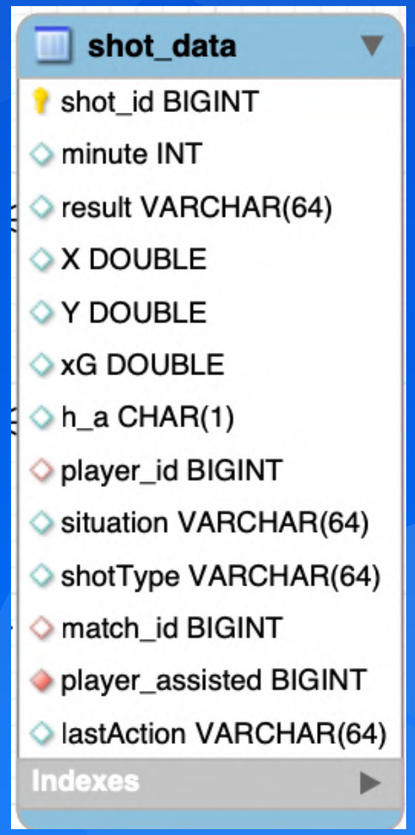
- Holds players data with changing data between seasons.
- Derived from other tables during normalization process.

```sql
-- Table 2: Player seasonal statistics (attributes that change each season)
CREATE TABLE fut23 (
    player_id BIGINT,
    year INT,
    team_id BIGINT,
    -- League removed: derive from team_id
    Rating INT,
    Position VARCHAR(64),
    Run_type VARCHAR(64),
    Price VARCHAR(64),
    Skill INT,
    Weak_foot INT,
    Attack_rate CHAR(2),
    Defense_rate CHAR(2),
    Pace DOUBLE,
    Shoot DOUBLE,
    Pass DOUBLE,
    Drible DOUBLE,
    Defense DOUBLE,
    Physical DOUBLE,
    Popularity INT,
    -- Base_Stats INT, -- REMOVED: calculated from Pace+Shoot+Pass+Drible+Defense+Physical
    -- In_Game_Stats INT, -- REMOVED: if calculated/derived
    time INT,
    goals INT,
    xG DOUBLE,
    assists INT,
    xA DOUBLE,
    yellow_cards INT,
    red_cards INT,
    npg INT,
    npxG DOUBLE,
    xGChain DOUBLE,
    xGBuildup DOUBLE,
    best_shot_id BIGINT,
    PRIMARY KEY (player_id, year),
    CONSTRAINT fk_player_seasons_player
        FOREIGN KEY (player_id)
        REFERENCES players(player_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT fk_player_seasons_team
        FOREIGN KEY (team_id)
        REFERENCES teams(team_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL,
    CONSTRAINT fk_player_seasons_best_shot
        FOREIGN KEY (best_shot_id)
        REFERENCES shot_data(shot_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Update shot_data foreign keys to reference the correct table
-- The player_id and player_assisted should reference the players table
ALTER TABLE shot_data
DROP FOREIGN KEY fk_shot_player,
DROP FOREIGN KEY fk_shot_assist_player;

ALTER TABLE shot_data
ADD CONSTRAINT fk_shot_player
    FOREIGN KEY (player_id)
    REFERENCES players(player_id)
    ON UPDATE CASCADE
    ON DELETE SET NULL,
ADD CONSTRAINT fk_shot_assist_player
    FOREIGN KEY (player_assisted)
    REFERENCES players(player_id)
    ON UPDATE CASCADE
    ON DELETE SET NULL;
```

ER-TO-RELATIONAL MAPPING

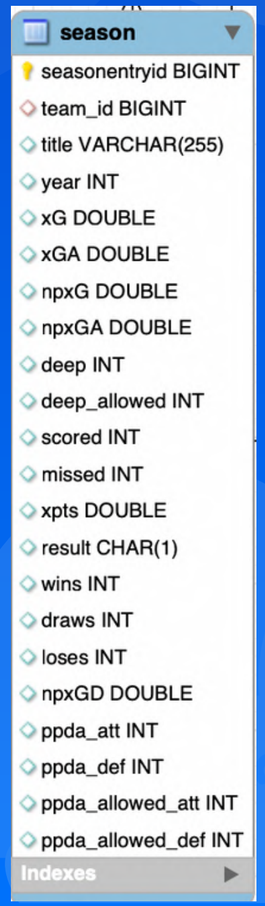


shot_data

- Includes shots from players.
- Changed during normalization.
- Connects to matches and players table.

```sql
-- CORRECTED: shot_data table - removed 'season' (derive from match_id)
CREATE TABLE shot_data (
    shot_id BIGINT PRIMARY KEY,
    minute INT,
    result VARCHAR(64),
    X DOUBLE,
    Y DOUBLE,
    xG DOUBLE,
    h_a CHAR(1),
    player_id BIGINT,
    situation VARCHAR(64),
    -- season INT, -- REMOVED: can be derived from match_id -> season
    shotType VARCHAR(64),
    match_id BIGINT,
    player_assisted BIGINT,
    lastAction VARCHAR(64),
    CONSTRAINT fk_shot_match
        FOREIGN KEY (match_id)
        REFERENCES match_info(match_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL,
    CONSTRAINT fk_shot_player
        FOREIGN KEY (player_id)
        REFERENCES players(player_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL,
    CONSTRAINT fk_shot_assist_player
        FOREIGN KEY (player_assisted)
        REFERENCES players(player_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

ER-TO-RELATIONAL MAPPING

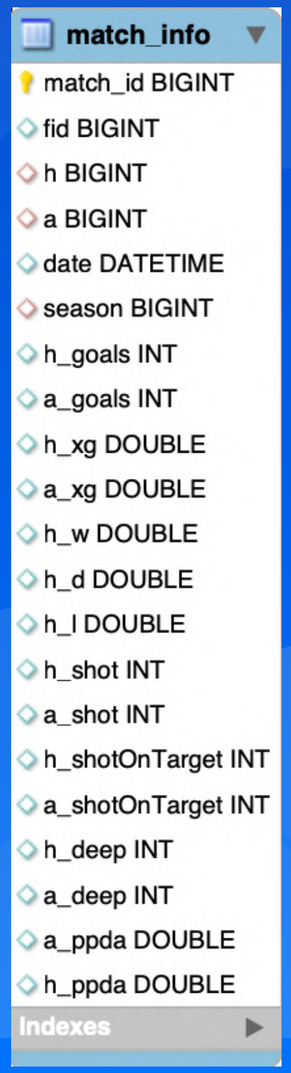



```sql
CREATE TABLE season (
    seasonentryid BIGINT PRIMARY KEY,
    team_id BIGINT,
    title VARCHAR(255),
    year INT,
    xG DOUBLE,
    xGA DOUBLE,
    npxG DOUBLE,
    npxGA DOUBLE,
    deep INT,
    deep_allowed INT,
    scored INT,
    missed INT,
    xpts DOUBLE,
    result CHAR(1),
    wins INT,
    draws INT,
    loses INT,
    -- pts INT,  -- REMOVED: calculated as (wins * 3 + draws * 1)
    npxGD DOUBLE,
    ppda_att INT,
    ppda_def INT,
    ppda_allowed_att INT,
    ppda_allowed_def INT,
    CONSTRAINT uq_season_team_year UNIQUE (team_id, year),
    CONSTRAINT fk_season_team
        FOREIGN KEY (team_id)
        REFERENCES teams(team_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

season

- Oversees season-long team performance metrics including xG, xGA, npxG, npxGA, points, wins, draws, and losses.
- Connects to teams table.
- Changed during normalization.

ER-TO-RELATIONAL MAPPING




```sql
CREATE TABLE match_info (
    match_id BIGINT PRIMARY KEY,
    fid BIGINT,
    h BIGINT,
    a BIGINT,
    date DATETIME,
    season BIGINT,
    h_goals INT,
    a_goals INT,
    h_xg DOUBLE,
    a_xg DOUBLE,
    h_w DOUBLE,
    h_d DOUBLE,
    h_l DOUBLE,
    -- league VARCHAR(128), -- REMOVED: can be derived from h or a team
    h_shot INT,
    a_shot INT,
    h_shotOnTarget INT,
    a_shotOnTarget INT,
    h_deep INT,
    a_deep INT,
    a_ppda DOUBLE,
    h_ppda DOUBLE,
    CONSTRAINT uq_match_home_away_date UNIQUE (h, a, date),
    CONSTRAINT fk_match_home_team
        FOREIGN KEY (h)
        REFERENCES teams(team_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL,
    CONSTRAINT fk_match_away_team
        FOREIGN KEY (a)
        REFERENCES teams(team_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL,
    CONSTRAINT fk_match_season
        FOREIGN KEY (season)
        REFERENCES season(seasonentryid)
        ON UPDATE CASCADE
        ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

match_info

- Holds match data with team and season foreign keys

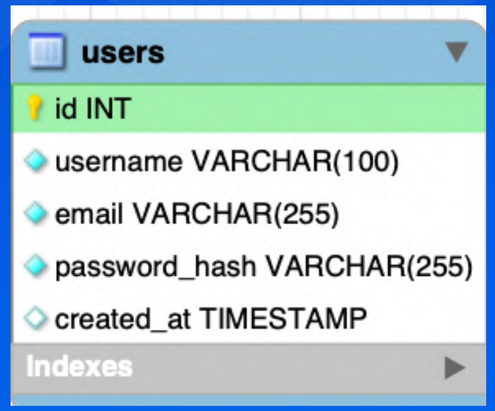
ER-TO-RELATIONAL MAPPING

teams

- Holds the central teams data that serves as a reference point for multiple other tables.
- League is added during normalization.

```sql
CREATE TABLE teams (
    team_id BIGINT PRIMARY KEY,
    team_name VARCHAR(255),
    league VARCHAR(128), -- Added: if team belongs to one league
    CONSTRAINT uq_teams_team_name UNIQUE (team_name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

ER-TO-RELATIONAL MAPPING



users

- Holds user data with not null requirement.
- Was not changed during normalization.

```sql
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_users_username UNIQUE (username),
    CONSTRAINT uq_users_email UNIQUE (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

COMPLEX QUERIES

```sql
SELECT
    -- Player's performance in similar situations
    COUNT(DISTINCT s.shot_id) as similar_shots_count,
    SUM(CASE WHEN s.result = 'Goal' THEN 1 ELSE 0 END) as similar_goals,
    AVG(s.xG) as avg_similar_xg,

    -- Team performance comparison
    AVG(CASE WHEN s.h_a = 'h' THEN m.h_xg ELSE m.a_xg END) as team_avg_match_xg,
    AVG(CASE WHEN s.h_a = 'h' THEN m.h_goals ELSE m.a_goals END) as team_avg_goals,

    -- Opposition defensive stats
    AVG(CASE WHEN s.h_a = 'h' THEN m.a_ppda ELSE m.h_ppda END) as opp_avg_ppda,
    AVG(CASE WHEN s.h_a = 'h' THEN sea.deep_allowed ELSE sea.deep END) as opp_deep_allowed,

    -- Player season form
    ps.goals as player_season_goals,
    ps.xG as player_season_xg,
    ps.npg as player_non_penalty_goals,
    ps.xGChain as player_xg_chain,

    -- League context
    COUNT(DISTINCT CASE WHEN league_shots.result = 'Goal' THEN league_shots.shot_id END) as league_similar_goals,
    COUNT(DISTINCT league_shots.shot_id) as league_similar_shots,
    AVG(league_shots.xG) as league_avg_similar_xg

FROM shot_data s

-- Join 1: Get match information
INNER JOIN match_info m ON s.match_id = m.match_id

-- Join 2: Get player season stats (NOW uses player_seasons table with composite key)
LEFT JOIN player_seasons ps ON
    s.player_id = ps.player_id
    AND YEAR(m.date) = ps.year  -- Derive year from match date since season was removed from shot_data

-- Join 3: Get team season defensive stats (opponent)
LEFT JOIN season sea ON
    CASE
        WHEN s.h_a = 'h' THEN m.a = sea.team_id
        ELSE m.h = sea.team_id
    END
    AND YEAR(m.date) = sea.year  -- Use match date to determine season year

-- Join 4: Self join to get league-wide similar shots
LEFT JOIN shot_data league_shots ON
    YEAR((SELECT m2.date FROM match_info m2 WHERE m2.match_id = league_shots.match_id)) = YEAR(m.date)  -- Compare seasons via match date
s   AND league_shots.situation = s.situation
    AND league_shots.shotType = s.shotType
    AND ABS(league_shots.xG - s.xG) < 0.1
    AND league_shots.shot_id != s.shot_id

WHERE s.shot_id = %s
    AND s.player_id = %s
    AND s.situation = (SELECT situation FROM shot_data WHERE shot_id = %s)
    AND s.shotType = (SELECT shotType FROM shot_data WHERE shot_id = %s)
    AND ABS(s.xG - (SELECT xG FROM shot_data WHERE shot_id = %s)) < 0.15

GROUP BY
    ps.goals, ps.xG, ps.npg, ps.xGChain;
```

COMPLEX QUERIES

🎯 Part 1: Player's Similar Shot Performance

```sql
SELECT
    -- Player's performance in similar situations
    COUNT(DISTINCT s.shot_id) AS similar_shots_count,
    SUM(CASE WHEN s.result = 'Goal'
        THEN 1 ELSE 0 END) AS similar_goals,
    AVG(s.xG) AS avg_similar_xg
```

What This Does:

- Counts total similar shots by this player
- Uses CASE to count only goals (result = 'Goal')
- Calculates average xG for these shots

EXAMPLE OUTPUT

52 shots | 16 goals | 0.24 avg xG

Why It Matters: Shows player's historical conversion rate (16/52 = 30.8%) compared to expected rate (0.24 × 52 = 12.5 expected goals)

COMPLEX QUERIES

Part 2: Team Performance Context

SQL CODE

```sql
-- Team performance comparison
AVG(CASE WHEN s.h_a = 'h'
    THEN m.h_xg
    ELSE m.a_xg END) AS team_avg_match_xg,

AVG(CASE WHEN s.h_a = 'h'
    THEN m.h_goals
    ELSE m.a_goals END) AS team_avg_goals
```

What This Does:

- Uses h_a field to identify home or away
- Selects correct team's xG based on location
- Averages across all similar shot situations

EXAMPLE OUTPUT

1.8 avg xG | 1.5 avg goals

Why It Matters: If h_a = 'h', get home stats (h_xg, h_goals). If away, get away stats. Shows team's overall offensive strength.

COMPLEX QUERIES

Part 3: Opposition Defensive Quality

SQL CODE

```sql
-- Opposition defensive stats
AVG(CASE WHEN s.h_a = 'h'
    THEN m.a_ppda
    ELSE m.h_ppda END) AS opp_avg_ppda,

AVG(CASE WHEN s.h_a = 'h'
    THEN sea.deep_allowed
    ELSE sea.deep END) AS opp_deep_allowed
```

What This Does:

- Gets OPPOSING team's defensive metrics
- If home (h), get away team stats (a_ppda)
- PPDA = Passes Per Defensive Action

EXAMPLE OUTPUT

10.5 PPDA | 8.2 deep allowed

Why It Matters: Lower PPDA = aggressive pressing (harder to score). Higher deep_allowed = weak defense (easier to score).

COMPLEX QUERIES

Part 4: Recent Form Subquery (Last 30 Days)

SQL CODE

```sql
(SELECT COUNT(CASE WHEN recent.result = 'Goal'
             THEN 1 END)
 FROM shot_data recent
 INNER JOIN match_info m_recent
    ON recent.match_id = m_recent.match_id
 WHERE recent.player_id = s.player_id
   AND m_recent.date < m.date
   AND m_recent.date >= DATE_SUB(m.date,
                        INTERVAL 30 DAY)
) AS recent_goals_last_30days
```

What This Does:

- Subquery looks at player's recent performance
- date < m.date ensures we only look BEFORE this shot
- DATE_SUB gets dates in last 30 days
- Counts goals in that window

EXAMPLE OUTPUT

6 goals in last 30 days

Why It Matters: Player on hot streak (6 goals) vs cold streak (0 goals) = different confidence level. Recent form predicts immediate performance.

COMPLEX QUERIES

Part 5: Team Ranking Subquery

SQL CODE

```
(SELECT COUNT(DISTINCT ps2.player_id) + 1
 FROM player_seasons ps2
 INNER JOIN shot_data s2
    ON s2.player_id = ps2.player_id
 INNER JOIN match_info m2
    ON s2.match_id = m2.match_id
 WHERE ps2.team_id = ps.team_id
   AND ps2.year = ps.year
   AND s2.situation = s.situation
   AND ps2.goals > ps.goals
) AS team_rank_by_goals_in_situation
```

What This Does:

- Counts teammates with MORE goals
- Filters by same team_id and year
- Only counts goals in same situation
- +1 gives player's actual rank

EXAMPLE OUTPUT

Rank #1 (Best scorer)

Why It Matters: If 0 teammates have more goals, COUNT = 0, so rank = 0 + 1 = 1st place! Shows player's importance to team.

COMPLEX QUERIES

Part 6: Understanding the JOINs

shot_data (s) Base Table → match_info (m) Match Details → player_seasons (ps) Player Stats → season (sea) Opponent Defense → league_shots League Benchmark

```
JOIN 1: MATCH INFO
INNER JOIN match_info m
  ON s.match_id = m.match_id

JOIN 2: PLAYER SEASONS
LEFT JOIN player_seasons ps
  ON s.player_id = ps.player_id
  AND YEAR(m.date) = ps.year

JOIN 3: OPPONENT STATS
LEFT JOIN season sea ON
  CASE
    WHEN s.h_a = 'h' THEN m.a = sea.team_id
    ELSE m.h = sea.team_id
  END
  AND YEAR(m.date) = sea.year
```

How JOINs Work:

- **JOIN 1:** Get match date, teams, scores
- **JOIN 2:** Uses YEAR(m.date) since we removed season column
- **JOIN 3:** CASE statement picks OPPOSING team
- **JOIN 4:** Self-join finds similar shots league-wide

Key Insight: Each JOIN adds more context. Start with shot → add match info → add player stats → add opponent stats → compare to league.

COMPLEX QUERIES

Part 7: The WHERE Clause Filter

SQL CODE

```
WHERE s.shot_id = %s
  AND s.player_id = %s
  AND s.situation =
      (SELECT situation
       FROM shot_data
       WHERE shot_id = %s)
  AND s.shotType =
      (SELECT shotType
       FROM shot_data
       WHERE shot_id = %s)
  AND ABS(s.xG -
      (SELECT xG
       FROM shot_data
       WHERE shot_id = %s)) < 0.15
```

What This Does:

- Filters to specific shot being analyzed
- Only includes shots by same player
- Matches situation (OpenPlay, SetPiece, etc.)
- Matches shotType (RightFoot, Header, etc.)
- ABS ensures xG within 0.15 range

EXAMPLE FILTER

Shot: OpenPlay, RightFoot, xG: 0.25 ± 0.15

Why Subqueries? Each subquery gets attributes from the target shot. This ensures we compare apples to apples (same type of shots).

COMPLEX QUERIES

🎯 Shot Context & Comparison

Similar Situations

Similar Shots	Goals from Similar
1	0
Conversion Rate	**Avg xG (Similar)**
0.0%	0.099

Team Performance

Avg Match xG	Avg Goals per Match
1.48	2.00

Opposition Defense

Avg PPDA	Deep Allowed
21.41	4.00

League-Wide Comparison

League Similar Shots	League Goals
1523	71
League Conversion	**League Avg xG**
4.7%	0.051

COMPLEX QUERIES

Top Performing Players of the Match

```sql
SELECT
    p.player_id,
    COALESCE(p.player_name, s.player) AS player_name,
    p.position,
    CASE
        WHEN s.h_a = 'h' THEN COALESCE(th.team_name, mi.team_h)
        WHEN s.h_a = 'a' THEN COALESCE(ta.team_name, mi.team_a)
        ELSE COALESCE(s.h_team, s.a_team)
    END AS team,
    COUNT(s.shot_id) AS shots_taken,
    SUM(s.xG) AS total_xg,
    SUM(CASE WHEN s.result = 'Goal' THEN 1 ELSE 0 END) AS goals_scored,
    AVG(s.xG) AS avg_xg_per_shot,
    p.goals AS season_goals,
    p.assists AS season_assists,
    p.year AS season_year,
    (SELECT ROUND(SUM(CASE WHEN sd2.result = 'Goal' THEN 1 ELSE 0 END) * 100.0 / NULLIF(COUNT(*), 0), 1)
        FROM shot_data sd2
        WHERE sd2.player_id = p.player_id AND sd2.season = mi.season) AS season_conversion_rate
FROM match_info mi
INNER JOIN shot_data s ON mi.match_id = s.match_id -- inner join to filter only shots in this match
LEFT JOIN player p ON s.player_id = p.player_id AND p.year = mi.season
LEFT JOIN teams th ON mi.h = th.team_id
LEFT JOIN teams ta ON mi.a = ta.team_id
WHERE mi.match_id = %s
GROUP BY p.player_id, p.player_name, s.player, p.position, s.h_a, mi.team_h, mi.team_a, s.h_team, s.a_team, p.goals, p.assists, p.year, th.team_name, ta.team_name
HAVING shots_taken > 0
ORDER BY total_xg DESC, shots_taken DESC
LIMIT 10
```

COMPLEX QUERIES

Part 1: Player Identification & COALESCE

```sql
SELECT
    p.player_id,
    COALESCE(p.player_name, s.player) AS player_name,
    p.position
```

What This Does:

- Retrieves player_id as unique identifier
- COALESCE handles missing player names
- If p.player_name is NULL, use s.player instead
- Ensures we always have a name to display

EXAMPLE OUTPUT

ID: 12345 | Name: "Mohamed Salah" | Position: FW

Why COALESCE? Player data might exist in shot_data but not in player table (or vice versa). COALESCE ensures we always display a name by checking both sources.

COMPLEX QUERIES

Part 2: Team Name Determination

```sql
CASE
    WHEN s.h_a = 'h' THEN COALESCE(th.team_name, mi.team_h)
    WHEN s.h_a = 'a' THEN COALESCE(ta.team_name, mi.team_a)
END AS team
```

What This Does:

- Checks h_a field: 'h' = home, 'a' = away
- For home players: get team from teams table (th.team_name) or fallback to mi.team_h
- For away players: get team from teams table (ta.team_name) or fallback to mi.team_a

> EXAMPLE LOGIC
>
> **If h_a='h' → "Liverpool" | If h_a='a' → "Manchester City"**

COMPLEX QUERIES

🎯 Part 3: Shot Performance Metrics

SQL CODE

```sql
COUNT(s.shot_id) AS shots_taken,
SUM(s.xG) AS total_xg,
SUM(CASE WHEN s.result = 'Goal'
    THEN 1 ELSE 0 END) AS goals_scored,
AVG(s.xG) AS avg_xg_per_shot
```

What This Does:

- **COUNT(s.shot_id):** Total number of shots in this match
- **SUM(s.xG):** Combined expected goals (quality of chances)
- **SUM(CASE...):** Counts only shots where result = 'Goal'
- **AVG(s.xG):** Average quality per shot

EXAMPLE OUTPUT

8 shots | 2.4 total xG | 3 goals | 0.30 avg xG

Performance Insight: If goals_scored (3) > total_xg (2.4), player is outperforming expectations! If reversed, they're underperforming. avg_xg_per_shot shows shot quality.

COMPLEX QUERIES

Part 4: Season-Wide Context

SQL CODE

```
p.goals AS season_goals,
p.assists AS season_assists,
p.year AS season_year
```

What This Does:

- Pulls player's total season statistics from player table
- Provides context beyond just this match
- Helps identify if player is key contributor

EXAMPLE OUTPUT

Season: 25 goals | 12 assists | Year: 2022

Why This Matters: Comparing match performance (3 goals) to season totals (25 goals) shows if this is a typical or exceptional performance. A player with 1 season goal scoring 2 in this match is huge!

COMPLEX QUERIES

Part 5: Season Conversion Rate Subquery

SQL CODE

```sql
(SELECT ROUND(
    SUM(CASE WHEN sd2.result = 'Goal'
        THEN 1 ELSE 0 END) * 100.0
    / NULLIF(COUNT(*), 0), 1)
 FROM shot_data sd2
 WHERE sd2.player_id = p.player_id
   AND sd2.season = mi.season
) AS season_conversion_rate
```

What This Does:

- Subquery calculates goals/shots percentage for entire season
- Filters to same player (sd2.player_id = p.player_id)
- Filters to same season (sd2.season = mi.season)
- NULLIF prevents division by zero
- Multiplies by 100.0 to get percentage
- ROUND(..., 1) gives one decimal place

EXAMPLE CALCULATION

25 goals / 120 shots = 20.8%

Why This Matters: If match conversion (3/8 = 37.5%) exceeds season rate (20.8%), player is having an exceptional match! Elite strikers typically convert 15-25% of shots.

COMPLEX QUERIES

Part 6: Understanding the 4-Table JOIN Structure

match_info (mi) Base Table → shot_data (s) INNER JOIN → player (p) LEFT JOIN → teams (th/ta) LEFT JOIN x2

```
JOIN 1: MATCH → SHOTS (INNER)
FROM match_info mi
INNER JOIN shot_data s
    ON mi.match_id = s.match_id

JOIN 2: SHOTS → PLAYER (LEFT)
LEFT JOIN player p
    ON s.player_id = p.player_id
    AND p.year = mi.season

JOIN 3 & 4: MATCH → TEAMS (LEFT X2)
LEFT JOIN teams th ON mi.h = th.team_id
LEFT JOIN teams ta ON mi.a = ta.team_id
```

How JOINs Work:

- **INNER JOIN shot_data:** Only include matches with shot data (filters out matches with no shots)
- **LEFT JOIN player:** Keep all shots even if player details missing
- **Additional condition:** p.year = mi.season ensures correct season data
- **LEFT JOIN teams (twice):** th for home team, ta for away team
- **mi.h and mi.a:** Foreign keys to team_id in teams table

Key Insight: INNER JOIN on shot_data means "only show if shots exist." LEFT JOINs on player and teams mean "show shots even if those details are missing." This prevents losing shot data due to incomplete player or team records.

Part 7: WHERE Clause - Match Filter

```
SQL CODE
WHERE mi.match_id = %s
```

What This Does:

- Filters entire query to one specific match
- %s is a placeholder replaced at runtime
- Example: WHERE mi.match_id = 2068

EFFECT

Only analyzes shots from match #2068

COMPLEX QUERIES

Part 8: GROUP BY - Aggregation Logic

SQL CODE

```
GROUP BY
    p.player_id,
    p.player_name,
    s.player,
    p.position,
    s.h_a,
    mi.team_h,
    mi.team_a,
    s.h_team,
    s.a_team,
    p.goals,
    p.assists,
    p.year,
    th.team_name,
    ta.team_name
```

What This Does:

- Groups shots by unique player combinations
- All non-aggregated SELECT columns must be in GROUP BY
- MySQL ONLY_FULL_GROUP_BY mode enforces this rule
- Ensures consistent grouping across all fields

RESULT

One row per player with aggregated shot stats

Why So Many Fields? Every column in SELECT that's not inside an aggregate function (COUNT, SUM, AVG) MUST be in GROUP BY. This includes CASE statement components (h_a, team names) and fallback values (s.player).

COMPLEX QUERIES

Part 9: HAVING, ORDER BY & LIMIT

SQL CODE

```sql
HAVING shots_taken > 0
ORDER BY total_xg DESC, shots_taken DESC
LIMIT 10
```

What This Does:

- **HAVING shots_taken > 0:** Filters out players with no shots (after grouping)
- **ORDER BY total_xg DESC:** Sorts by highest expected goals first
- **Then shots_taken DESC:** Tie-breaker: more shots = higher rank
- **LIMIT 10:** Only return top 10 performers

EXAMPLE RANKING

1. Salah (2.8 xG) → 2. Kane (2.5 xG) → 3. Haaland (2.5 xG, 9 shots)

Why This Order? total_xg (expected goals) indicates shot quality and quantity combined. A player with 2.5 xG from 8 shots > player with 0.5 xG from 15 shots. The first player creates better chances.

COMPLEX QUERIES

⭐ Top Performers in This Match

Players with highest expected goals (xG)

PLAYER	POSITION	TEAM	SHOTS	TOTAL XG	GOALS	AVG XG/SHOT	SEASON STATS	SEASON CONV%
Vinícius Júnior	F M S	Real Madrid	6	2.01	1	0.335	*15G / 5A*	19.2%
Lucas Vázquez	D S	Real Madrid	1	0.30	1	0.297	*3G / 5A*	27.3%
Andreas Christensen	D M S	Barcelona	2	0.17	1	0.083	*2G / 2A*	18.2%
Luka Modric	M S	Real Madrid	2	0.10	0	0.051	*2G / 6A*	8.0%
Robert Lewandowski	F S	Barcelona	2	0.10	0	0.050	*19G / 8A*	20.7%
João Félix	F M S	Barcelona	2	0.09	0	0.046	*7G / 3A*	11.5%
Raphinha	F M S	Barcelona	1	0.06	0	0.062	*6G / 9A*	10.2%
Aurélien Tchouaméni	D M S	Real Madrid	1	0.02	0	0.015	*3G / 1A*	11.1%
Federico Valverde	M S	Real Madrid	1	0.01	0	0.013	*2G / 7A*	3.6%

COMPLEX QUERIES

```sql
SELECT
            t.team_id,
            t.team_name,
            s.year,
            -- Season summary
            MAX(s.pts) AS total_points,
            MAX(s.wins) AS wins,
            MAX(s.draws) AS draws,
            MAX(s.loses) AS loses,
            ROUND(AVG(s.xG), 2) AS avg_xG,
            ROUND(AVG(s.xGA), 2) AS avg_xGA,
            -- Match + Shot aggregated metrics
            COUNT(DISTINCT sd.shot_id) AS total_shots,
            SUM(CASE WHEN sd.result = 'Goal' THEN 1 ELSE 0 END) AS total_goals,
            ROUND(SUM(sd.xG), 2) AS total_xg,
            ROUND(
                SUM(CASE WHEN sd.result = 'Goal' THEN 1 ELSE 0 END) / NULLIF(COUNT(sd.shot_id), 0),
               3
            ) AS conversion_rate,
            -- Nested query: Top scorer name (via player table)
           (
                SELECT p2.player_name
                FROM player p2
                WHERE p2.team_id = t.team_id
                  AND p2.year = s.year
                ORDER BY p2.goals DESC
                LIMIT 1
            ) AS top_scorer,
            -- Nested query: league-wide average xG (for comparison)
           (
                SELECT ROUND(AVG(sd2.xG), 3)
                FROM shot_data sd2
                INNER JOIN match_info mi2 ON sd2.match_id = mi2.match_id
                WHERE mi2.season = s.year
            ) AS league_avg_xg
        FROM teams t
        LEFT JOIN season s ON s.team_id = t.team_id
        LEFT JOIN match_info mi ON mi.team_h_id = t.team_id OR mi.team_a_id = t.team_id
        LEFT JOIN shot_data sd ON sd.match_id = mi.match_id
        WHERE s.year IS NOT NULL
        {}
        GROUP BY t.team_id, t.team_name, s.year
        ORDER BY total_points DESC, avg_xG DESC
        LIMIT %s
```

COMPLEX QUERIES

1 Query Overview & Purpose

🎯 **Query Goal:** Advanced season analysis with team performance metrics, shot data, and top scorer information

Total Tables	JOIN Operations	Subqueries
4	4	2

Aggregations
10+

```
-- Main Query Structure (SQL Injection Safe) SELECT [aggregated metrics] FROM teams t
LEFT JOIN season s LEFT JOIN match_info mi LEFT JOIN shot_data sd WHERE s.year IS NOT
NULL {} GROUP BY t.team_id, t.team_name, s.year ORDER BY total_points DESC LIMIT %s
```

🔒 **Security:** Uses parameterized queries with %s placeholders - safe from SQL injection

COMPLEX QUERIES

2 JOIN Structure & Table Relationships

teams (t)
team_id
team_name

→ LEFT JOIN

season (s)
team_id
year, pts, wins

→ LEFT JOIN

match_info (mi)
match_id
team_h_id, team_a_id

→ LEFT JOIN

shot_data (sd)
shot_id, match_id
xG, result

```
-- JOIN 1: Teams to Season LEFT JOIN season s ON s.team_id = t.team_id -- JOIN 2: Teams to Matches (home or away) LEFT JOIN match_info mi ON mi.team_h_id = t.team_id OR mi.team_a_id = t.team_id -- JOIN 3: Matches to Shot Data LEFT JOIN shot_data sd ON sd.match_id = mi.match_id
```

💡 **Why LEFT JOIN?** Preserves all teams even if they don't have season data, matches, or shots. Ensures complete dataset.

COMPLEX QUERIES

3 Aggregation Functions & Metrics

📊 **Grouped By:** team_id, team_name, year

```sql
-- Season Summary Metrics MAX(s.pts) AS total_points, MAX(s.wins) AS wins, MAX(s.draws)
AS draws, MAX(s.loses) AS loses, ROUND(AVG(s.xG), 2) AS avg_xG, ROUND(AVG(s.xGA), 2) AS
avg_xGA, -- Shot-Level Aggregations COUNT(DISTINCT sd.shot_id) AS total_shots, SUM(CASE
WHEN sd.result = 'Goal' THEN 1 ELSE 0 END) AS total_goals, ROUND(SUM(sd.xG), 2) AS
total_xg, -- Calculated Metric: Conversion Rate (SQL Injection Safe) ROUND( SUM(CASE
WHEN sd.result = 'Goal' THEN 1 ELSE 0 END) / NULLIF(COUNT(sd.shot_id), 0), 3 ) AS
conversion_rate
```

MAX Functions	AVG Functions	SUM Functions
6	2	3

COUNT Functions
2

COMPLEX QUERIES

4 Nested Subqueries (Correlated & Independent)

🔍 **Subquery 1:** Correlated - Finds top scorer for each team

```
-- Nested Query 1: Top Scorer (Correlated) ( SELECT p2.player_name FROM player p2 WHERE
p2.team_id = t.team_id AND p2.year = s.year ORDER BY p2.goals DESC LIMIT 1 ) AS
top_scorer
```

🔍 **Subquery 2:** Independent - Calculates league-wide average xG

```
-- Nested Query 2: League Average (Independent) ( SELECT ROUND(AVG(sd2.xG), 3) FROM
shot_data sd2 INNER JOIN match_info mi2 ON sd2.match_id = mi2.match_id WHERE mi2.season
= s.year ) AS league_avg_xg
```

Subquery Type	Executes Per	Subquery Type
Correlated	**Row**	**Independent**

Executes Per
Query

COMPLEX QUERIES

5 Complete Query Execution Flow

Execution Order:

1. FROM teams t - Start with base table
2. LEFT JOIN season s - Add season data
3. LEFT JOIN match_info mi - Add match data
4. LEFT JOIN shot_data sd - Add shot details
5. WHERE s.year IS NOT NULL {} - Filter valid seasons (parameterized)
6. GROUP BY team_id, team_name, year - Aggregate data
7. Execute subqueries for each row - Get top scorer & league avg
8. ORDER BY total_points DESC - Sort results
9. LIMIT - Return top N results

Final Output Columns:

- Team Info: team_id, team_name, year
- Season Stats: total_points, wins, draws, loses, avg_xG, avg_xGA
- Shot Metrics: total_shots, total_goals, total_xg, conversion_rate
- Advanced: top_scorer (subquery), league_avg_xg (subquery)

Security Implementation:

- **Parameterized Queries:** Uses %s placeholders for all user inputs
- **Safe String Formatting:** .format() only inserts SQL syntax, not user data
- **Input Validation:** limit is validated with min/max bounds (1-50)
- **Separate Parameter Binding:** User inputs passed via params list
- **No Direct Concatenation:** User data never directly inserted into SQL string

COMPLEX QUERIES

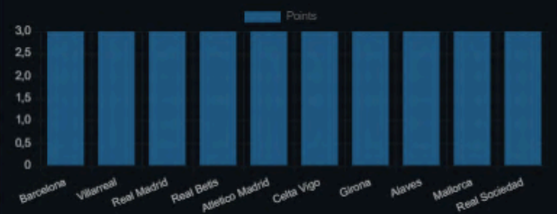
Seasons Advanced Analysis

← Back to Seasons | 2024 | Load

Aggregated team insights using Season + Match + Shot + Player tables

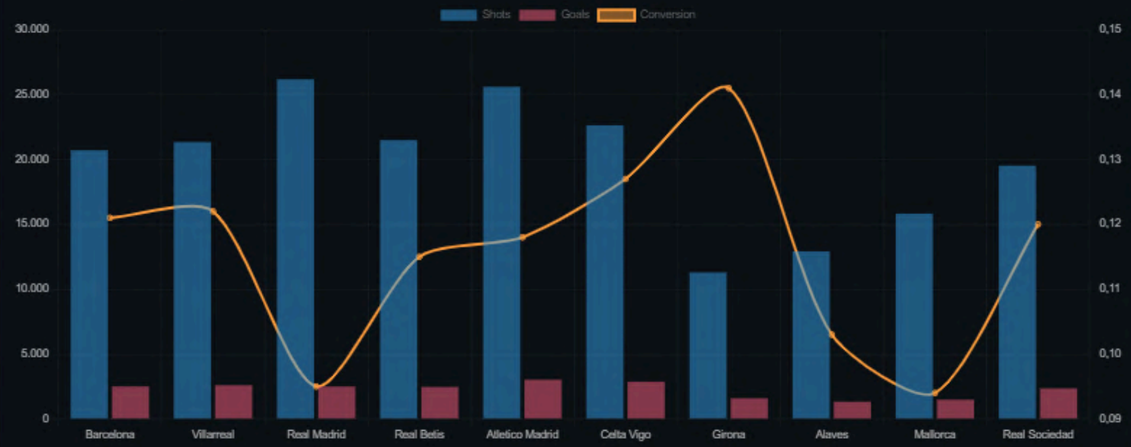
Selected Season	Top Team (Points)	Best Conversion	League Avg Shot xG
2024	**Barcelona**	**Girona**	—
Data aggregated across multiple tables	Points: 3 | Avg xG: 2.76 | Avg xGA: 1.30	Conv: 0.141 | Goals: 1592 | Shots: 11288	Average xG per shot for the selected season

Total Points (Top Teams)



Avg xG vs Avg xGA (Top Teams)

Shots vs Goals (Conversion)



Team Breakdown

Team	Points	W	D	L	Avg xG	Avg xGA	Goals	Shots	Total xG	Conv.	Top Scorer
Barcelona	3	1	0	1	2.76	1.30	2504	20712	2682.60	0.121	—
Villarreal	3	1	1	1	2.28	1.63	2600	21336	2661.10	0.122	—
Real Madrid	3	1	1	0	2.26	0.87	2496	26176	2835.08	0.095	—
Real Betis	3	1	1	1	1.85	1.02	2464	21488	2494.21	0.115	—
Atletico Madrid	3	1	1	0	1.71	0.84	3024	25608	3011.76	0.118	—
Celta Vigo	3	1	1	1	1.64	1.46	2872	22632	2789.45	0.127	—
Girona	3	1	1	1	1.52	0.99	1592	11288	1518.90	0.141	—
Alaves	3	1	1	1	1.47	1.32	1328	12920	1557.84	0.103	—
Mallorca	3	1	1	1	1.31	1.02	1485	15831	1623.22	0.094	—
Real Sociedad	3	1	1	1	1.29	1.52	2352	19520	2393.68	0.120	—
Sevilla	3	1	1	1	1.24	1.33	2456	23992	2680.11	0.102	—
Athletic Club	3	1	1	1	1.12	1.16	2280	22656	2671.37	0.101	—
Osasuna	3	1	1	1	1.10	1.35	1872	16992	1974.91	0.110	—
Espanyol	3	1	1	1	1.02	2.21	1917	16290	1921.53	0.118	—
Getafe	3	1	1	1	1.02	1.07	2112	19880	2261.14	0.106	—
Rayo Vallecano	3	1	1	1	0.98	1.19	1472	14432	1445.94	0.102	—
Valencia	3	1	1	1	0.80	1.75	1737	16929	1971.16	0.103	—
Real Valladolid	3	1	1	1	0.73	1.92	888	9664	1113.56	0.092	—
Las Palmas	1	0	1	1	1.08	2.44	776	6328	803.43	0.123	—

COMPLEX QUERIES

Players

```
"""
COMPLEX QUERY 5: All Features Combined - Nested Query, Complex Join (4+ tables), GROUP BY, Outer Join

This query demonstrates:
- Nested Query: Uses a correlated subquery to calculate average goals for players in the same position and year
- Complex Join (4+ tables): Joins player, fut23, teams, match_info, match_data, and shot_data (6 tables)
- GROUP BY: Groups by player attributes to aggregate statistics and calculate goals per game
- Outer Join (LEFT JOIN): Uses LEFT JOINs to include players even without FIFA ratings, team info, matches, or shot data

Returns top players with their performance metrics and position average goals for comparison.
"""
try:
    query = """
        SELECT
            p.player_name,
            t.team_name AS team_title,
            f.Position AS position,
            p.goals,
            p.assists,
            p.games,
            p.xG,
            CASE WHEN p.games > 0 THEN p.goals / p.games ELSE 0 END AS goals_per_game,
            (
                SELECT AVG(p2.goals)
                FROM player p2
                LEFT JOIN fut23 f2 ON p2.player_id = f2.player_id
                WHERE f2.Position = f.Position
                AND p2.year = p.year
                AND p2.games > 0
                AND f2.Position IS NOT NULL
                AND p2.goals IS NOT NULL
            ) AS position_avg_goals,
            f.Rating AS fifa_rating,
            f.Pace,
            f.Shoot,
            f.Pass,
            f.Defense,
            f.Physical,
            p.year,
            (
                SELECT COUNT(DISTINCT sd2.shot_id)
                FROM shot_data sd2
                WHERE sd2.player_id = p.player_id
            ) AS total_shots
        FROM player p
        LEFT JOIN fut23 f ON p.player_id = f.player_id
        LEFT JOIN teams t ON f.team_id = t.team_id
        LEFT JOIN match_info mi ON (mi.h = f.team_id OR mi.a = f.team_id) AND mi.season = p.year
        LEFT JOIN match_data md ON md.match_id = mi.match_id
        WHERE p.games > 0 AND p.goals > 0 AND f.Position IS NOT NULL
        GROUP BY p.season_player_id, p.player_id, p.player_name, p.games, p.goals, p.assists, p.xG,
                 f.Rating, f.Position, t.team_name, f.Pace, f.Shoot, f.Pass, f.Defense, f.Physical, p.year
        ORDER BY p.goals DESC, f.Rating DESC
        LIMIT 50
```

COMPLEX QUERIES

Part 1: Main SELECT - Player Statistics

SQL CODE

```sql
SELECT
    p.player_name,
    t.team_name AS team_title,
    f.Position AS position,
    p.goals,
    p.assists,
    p.games,
    p.xG
```

What This Does:

- Selects basic player information from player table (p)
- Gets team name from teams table (t) with alias
- Gets position from fut23 table (f)
- Retrieves key performance metrics: goals, assists, games, xG

EXAMPLE OUTPUT

Player: "Messi" | Team: "Barcelona" | Goals: 25

Why It Matters: This is the foundation - we're gathering all the core stats we need to analyze player performance and compare it to position averages.

Part 2: CASE Statement - Goals Per Game Calculation

SQL CODE

```sql
CASE WHEN p.games > 0
    THEN p.goals / p.games
    ELSE 0
END AS goals_per_game
```

What This Does:

- Conditional calculation: divides goals by games
- Prevents division by zero error
- Returns 0 if player has no games played
- Creates a normalized metric for comparison

EXAMPLE OUTPUT

25 goals / 30 games = 0.83 goals/game

Why It Matters: Goals per game is a standardized metric that allows fair comparison between players regardless of how many games they've played. A player with 10 goals in 10 games (1.0) is more efficient than one with 20 goals in 30 games (0.67).

COMPLEX QUERIES

🎯 Part 3: Nested Subquery - Position Average Goals

SQL CODE

```sql
(
    SELECT AVG(p2.goals)
    FROM player p2
    LEFT JOIN fut23 f2 ON p2.player_id = f2.player_id
    WHERE f2.Position = f.Position
      AND p2.year = p.year
      AND p2.games > 0
      AND f2.Position IS NOT NULL
      AND p2.goals IS NOT NULL
) AS position_avg_goals
```

What This Does:

- Correlated subquery: references outer query (f.Position, p.year)
- Calculates average goals for players in same position and year
- Filters out players with no games or null values
- Provides benchmark for comparison

EXAMPLE OUTPUT

Average goals for ST in 2023: 12.5 goals

Why It Matters: This is a correlated subquery - it runs for EACH row in the main query, using that row's position and year to find the average. If a striker scored 25 goals and the position average is 12.5, they're performing 2x better than average!

🎲 Part 4: Second Nested Subquery - Total Shots Count

SQL CODE

```sql
(
    SELECT COUNT(DISTINCT sd2.shot_id)
    FROM shot_data sd2
    WHERE sd2.player_id = p.player_id
) AS total_shots
```

What This Does:

- Another correlated subquery counting shots per player
- Uses DISTINCT to avoid counting duplicate shot_ids
- Links to main query via player_id
- Aggregates data from shot_data table

EXAMPLE OUTPUT

Player has taken 145 total shots

Why It Matters: This shows shooting volume. Combined with goals, we can calculate conversion rate. A player with 25 goals from 145 shots (17.2%) is more clinical than one with 20 goals from 200 shots (10%).

COMPLEX QUERIES

Part 5: Complex JOINs - Connecting 6 Tables

player (p) Base Table → fut23 (f) FIFA Ratings → teams (t) Team Info → match_info (mi) Match Details → match_data (md) Match Stats

```
JOIN 1: FIFA RATINGS
LEFT JOIN fut23 f
  ON p.player_id = f.player_id

JOIN 2: TEAM INFORMATION
LEFT JOIN teams t
  ON f.team_id = t.team_id

JOIN 3: MATCH INFO
LEFT JOIN match_info mi
  ON (mi.h = f.team_id OR mi.a = f.team_id)
  AND mi.season = p.year

JOIN 4: MATCH DATA
LEFT JOIN match_data md
  ON md.match_id = mi.match_id
```

How JOINs Work:

- **JOIN 1:** Links player to FIFA ratings (LEFT JOIN = includes players without FIFA data)
- **JOIN 2:** Gets team name from team_id in fut23 table
- **JOIN 3:** Complex condition - matches if team is home (h) OR away (a), AND same season/year
- **JOIN 4:** Links to match statistics data

Key Insight: LEFT JOINs ensure we include players even if they don't have FIFA ratings, team info, or match data. The OR condition in JOIN 3 handles both home and away matches for

Part 6: WHERE Clause - Filtering Results

```
SQL CODE
WHERE p.games > 0
  AND p.goals > 0
  AND f.Position IS NOT NULL
```

What This Does:

- Filters to only players who have played games
- Only includes players who have scored at least 1 goal
- Requires position data to exist (for position average calculation)

EXAMPLE FILTER

Shows only goal-scorers with position data

Why These Filters? We want meaningful comparisons - players with no goals can't be compared on goal metrics, and we need position data to calculate position averages. This ensures quality results.

COMPLEX QUERIES

📦 Part 7: GROUP BY - Aggregating Data

SQL CODE

```
GROUP BY p.season_player_id, p.player_id, p.player_name,
         p.games, p.goals, p.assists, p.xG,
         f.Rating, f.Position, t.team_name,
         f.Pace, f.Shoot, f.Pass, f.Defense, f.Physical, p.year
```

What This Does:

- Groups rows by unique player-season combination
- Includes all non-aggregated columns in GROUP BY
- Prevents duplicate rows when multiple matches exist
- Ensures one row per player per season

EXAMPLE OUTPUT

One row per player, even if they played 30 matches

Why GROUP BY? Since we're joining match_info and match_data, a player with 30 matches would create 30 rows. GROUP BY consolidates this into one row per player with their season totals.

🏆 Part 8: ORDER BY and LIMIT - Sorting Results

SQL CODE

```
ORDER BY p.goals DESC, f.Rating DESC
LIMIT 50
```

What This Does:

- Primary sort: goals descending (highest first)
- Secondary sort: FIFA rating descending (tiebreaker)
- LIMIT restricts results to top 50 players
- Returns the best goal-scorers with highest ratings

EXAMPLE OUTPUT

Top 50 players by goals, then by FIFA rating

Why This Order? Goals are the primary metric, but FIFA rating breaks ties. If two players have 25 goals, the one with higher rating ranks first. LIMIT 50 keeps results manageable and focuses on elite performers.

COMPLEX QUERIES

FIFA Attributes Comparison

Compare FIFA attributes across top players

Select Players (max 5):

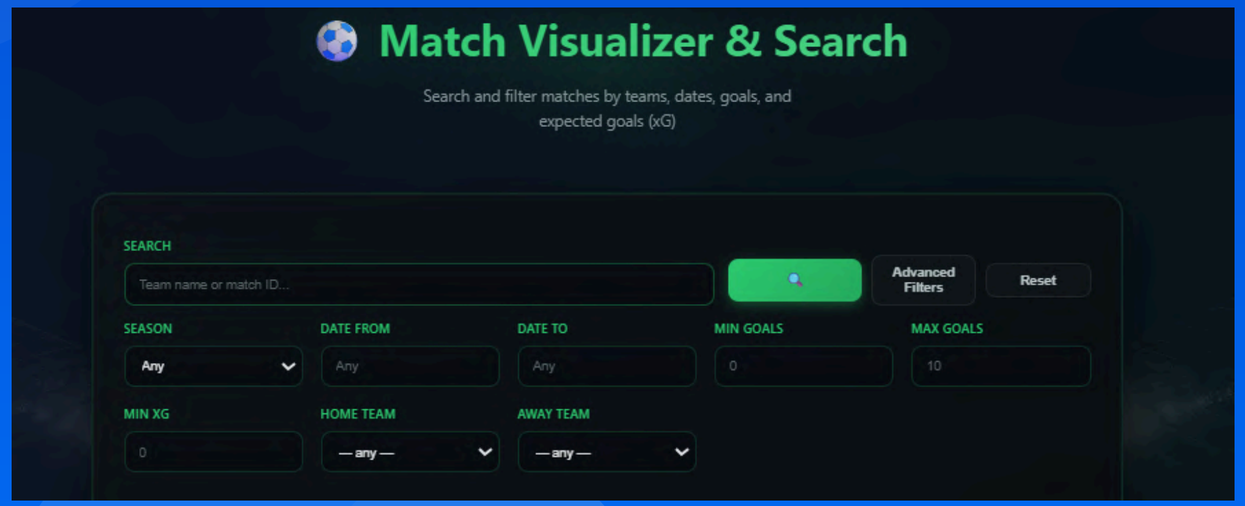
- [x] Alexander Sørloth
- [x] Robert Lewandowski
- [x] Ante Budimir
- [x] Antoine Griezmann
- [x] Youssef En-Nesyri
- [ ] Vinícius Júnior
- [ ] Borja Mayoral Moya
- [ ] Hugo Duro
- [ ] Jørgen Strand Larsen
- [ ] Iñaki Williams

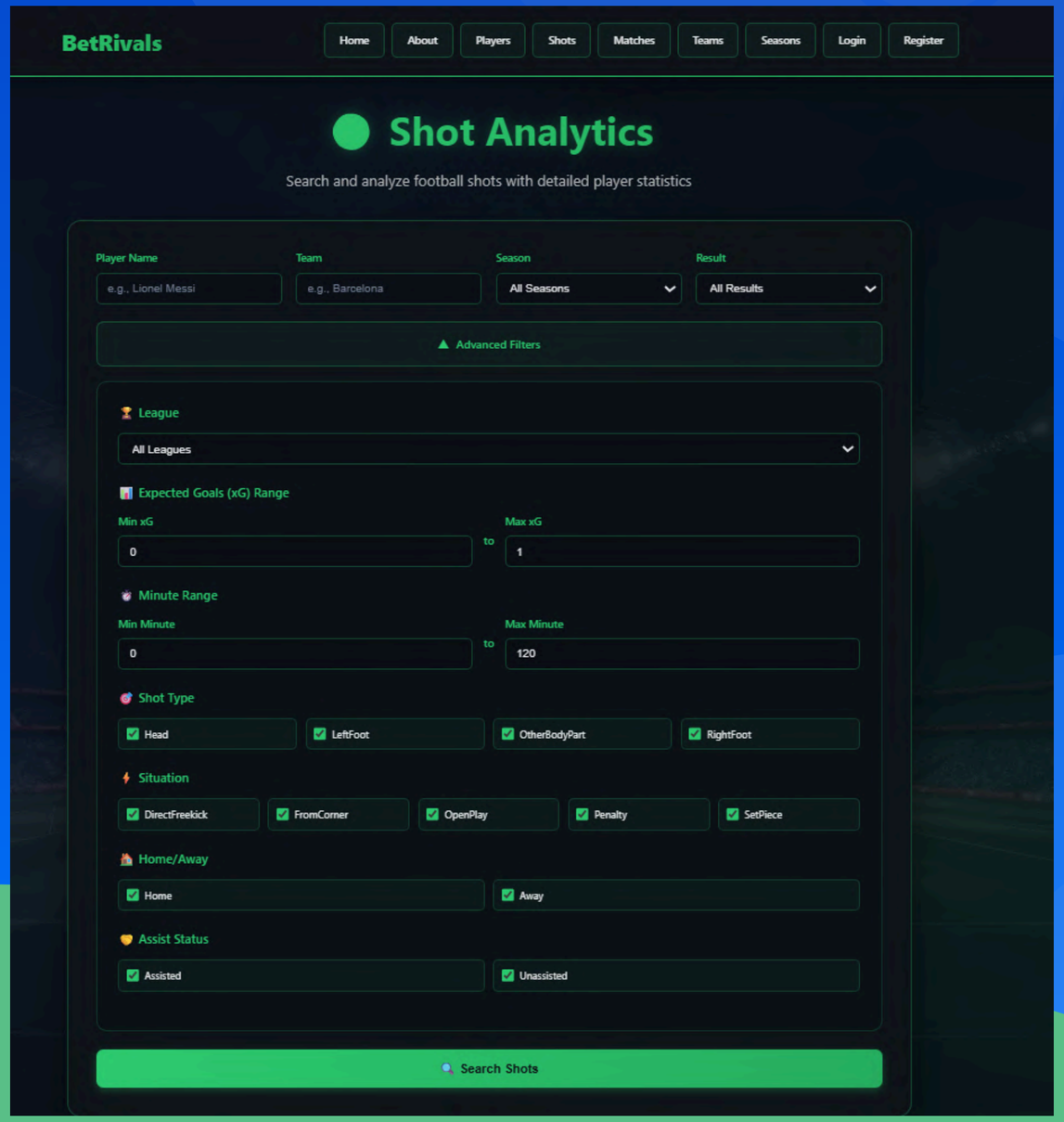
Top Performers by Goals

Top 20 players with position average reference line

FEATURES

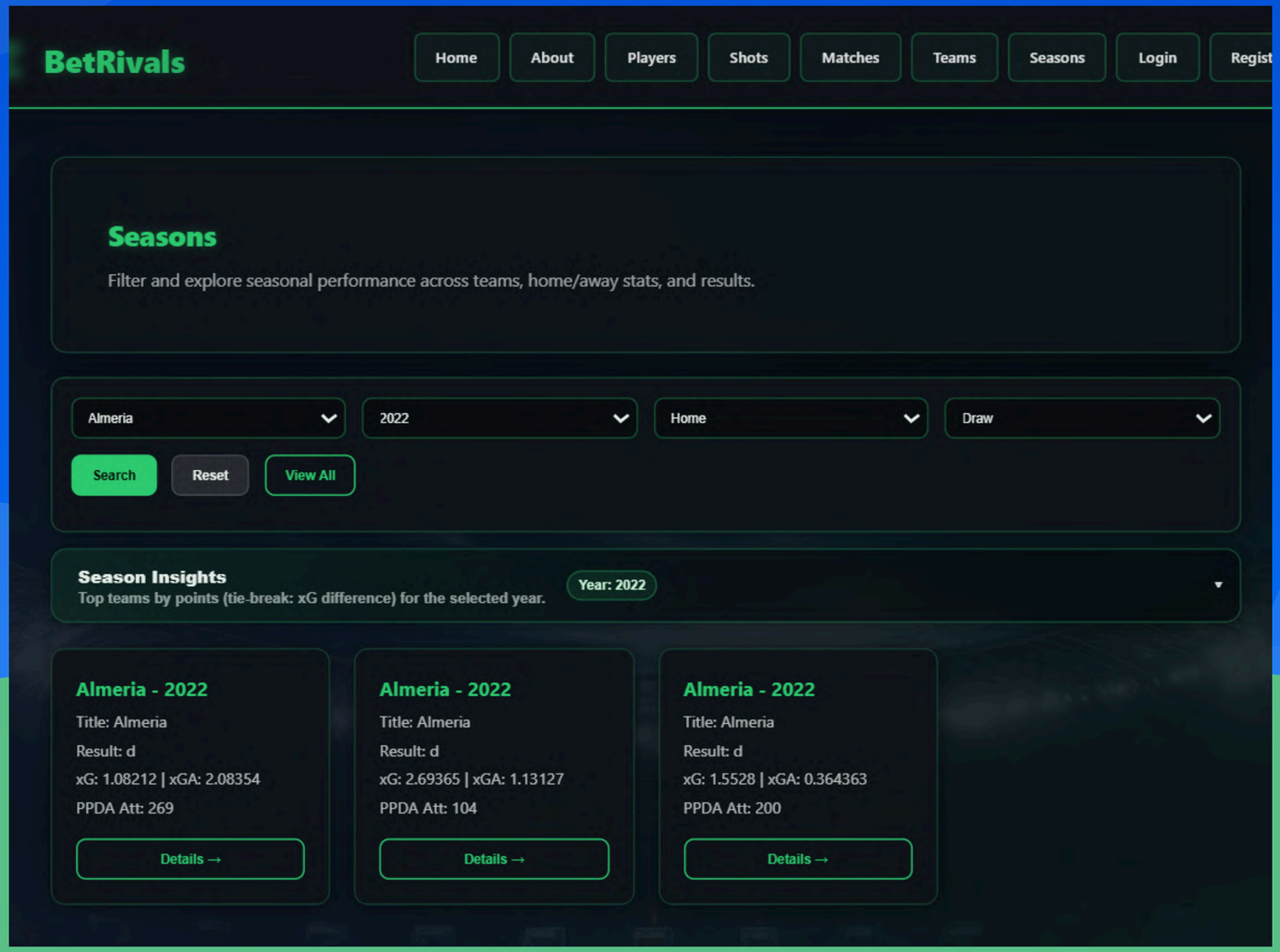
- ADVANCED FILTERING





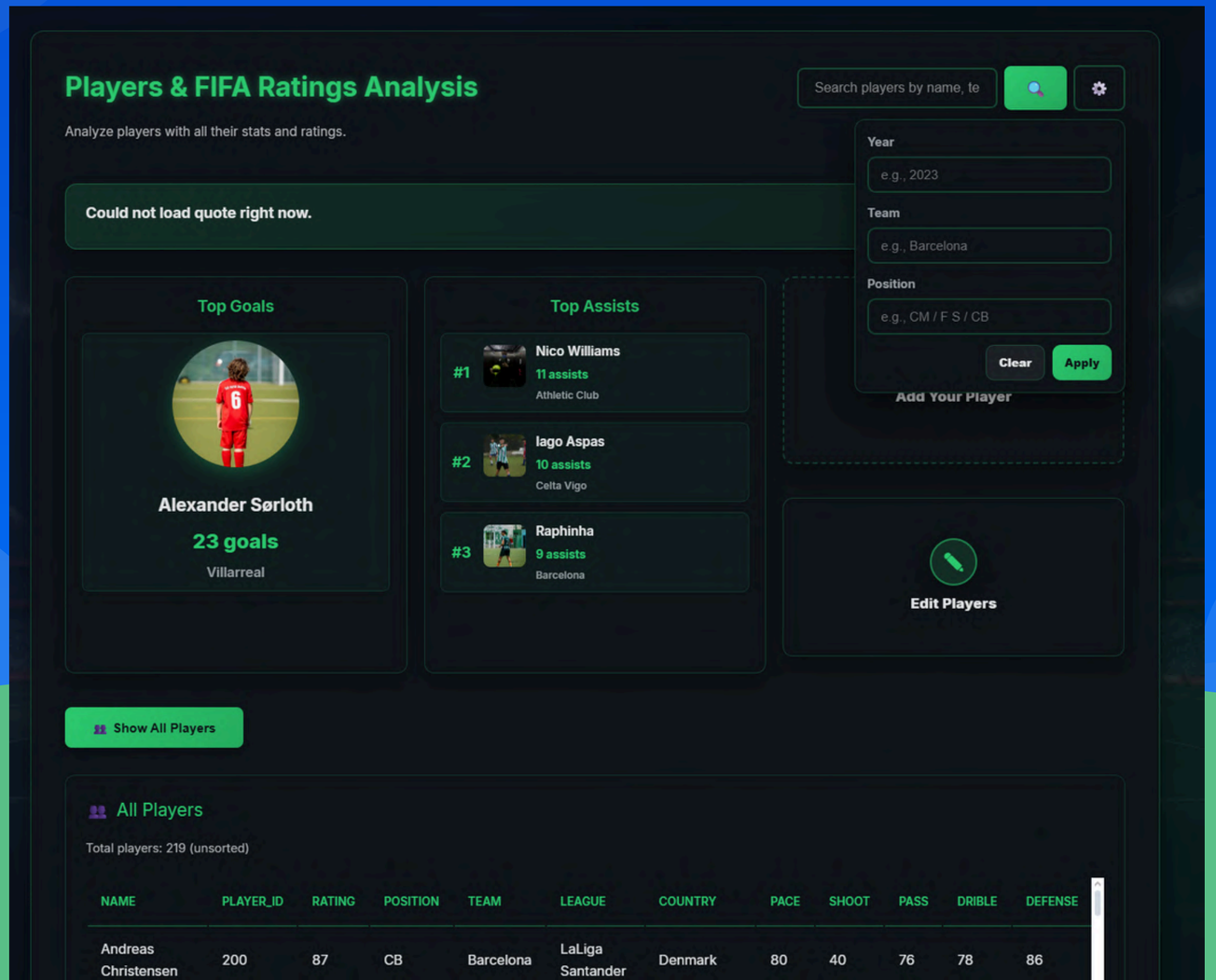
FEATURES

- ADVANCED FILTERING



FEATURES

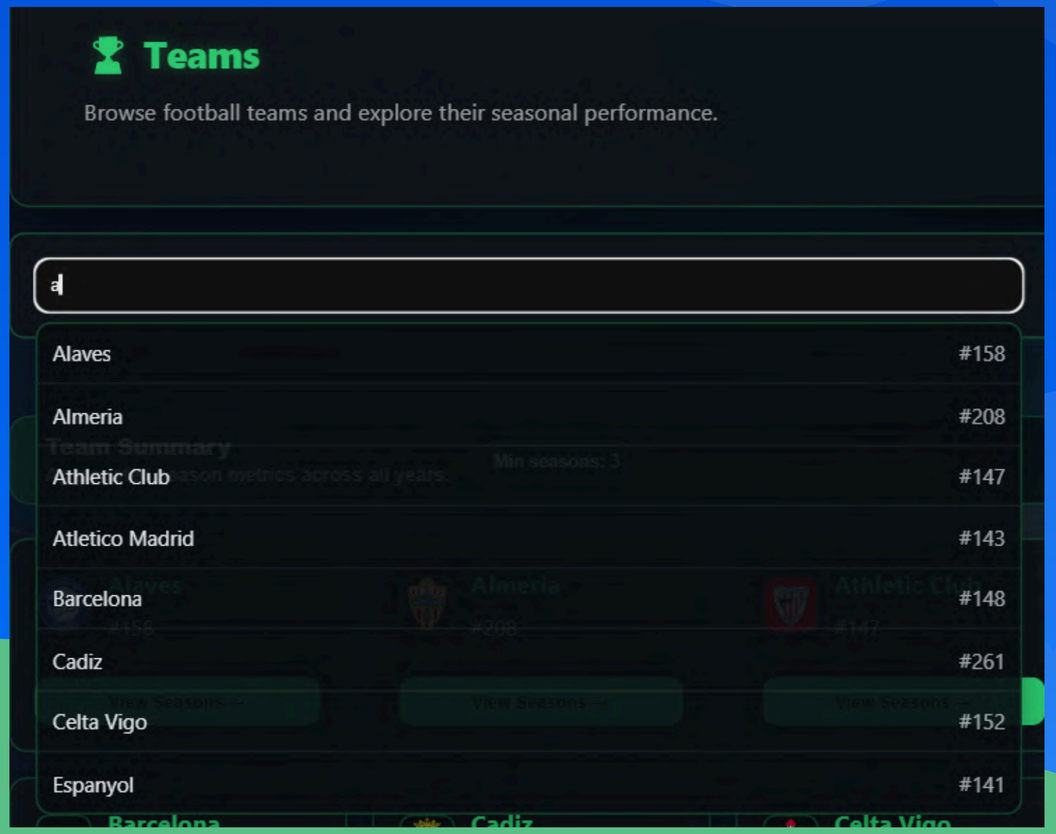
- ADVANCED FILTERING



FEATURES

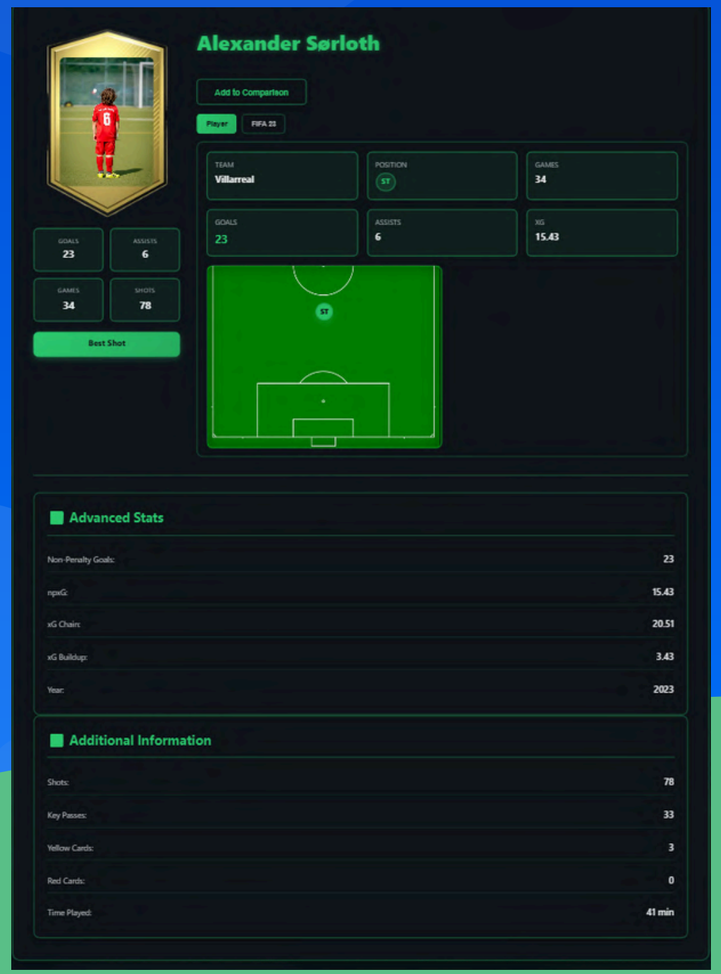
- AUTOCOMPLETE

```
192.168.0.133 - - [21/Dec/2025 17:36:40] "GET /api/admin/shots?page=1&limit=20&player=&result= HTTP/1.1" 200
192.168.0.133 - - [21/Dec/2025 17:36:40] "GET /api/admin/shots?page=1&limit=20&player=&result= HTTP/1.1" 200
192.168.0.133 - - [21/Dec/2025 17:36:43] "GET /api/admin/shots?page=2&limit=20&player=&result= HTTP/1.1" 200
192.168.0.133 - - [21/Dec/2025 17:36:45] "GET /api/admin/shots?page=3&limit=20&player=&result= HTTP/1.1" 200
```



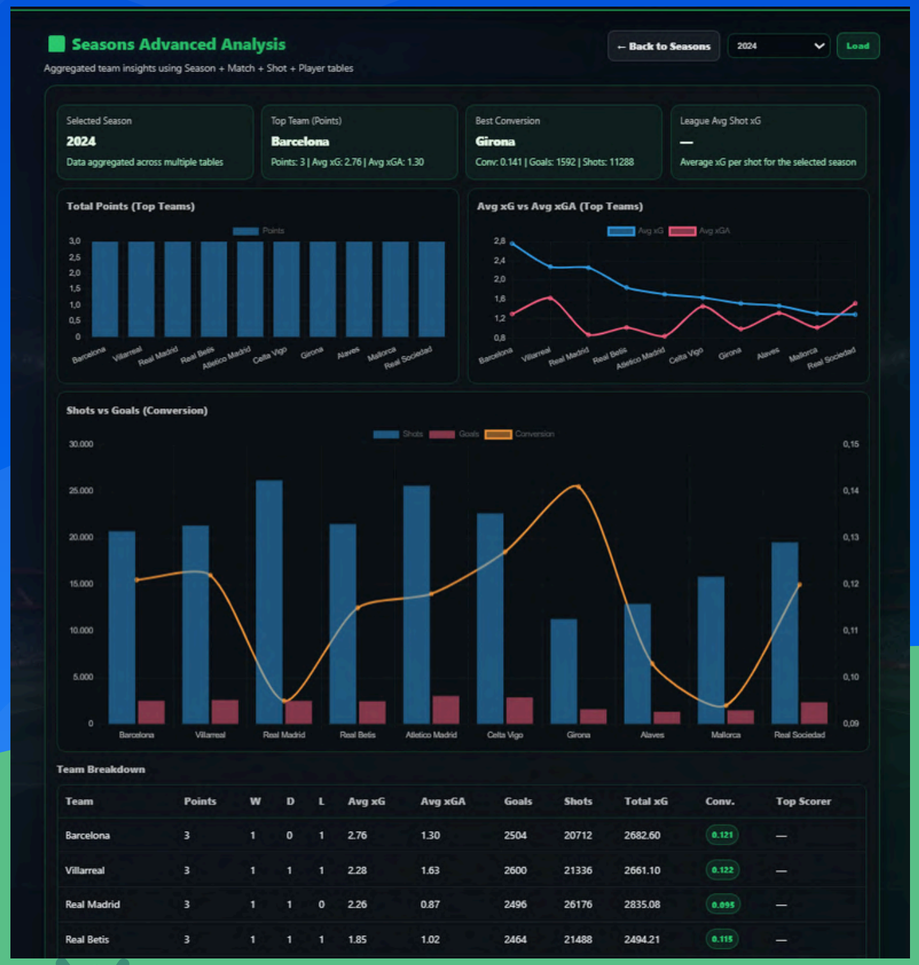
FEATURES

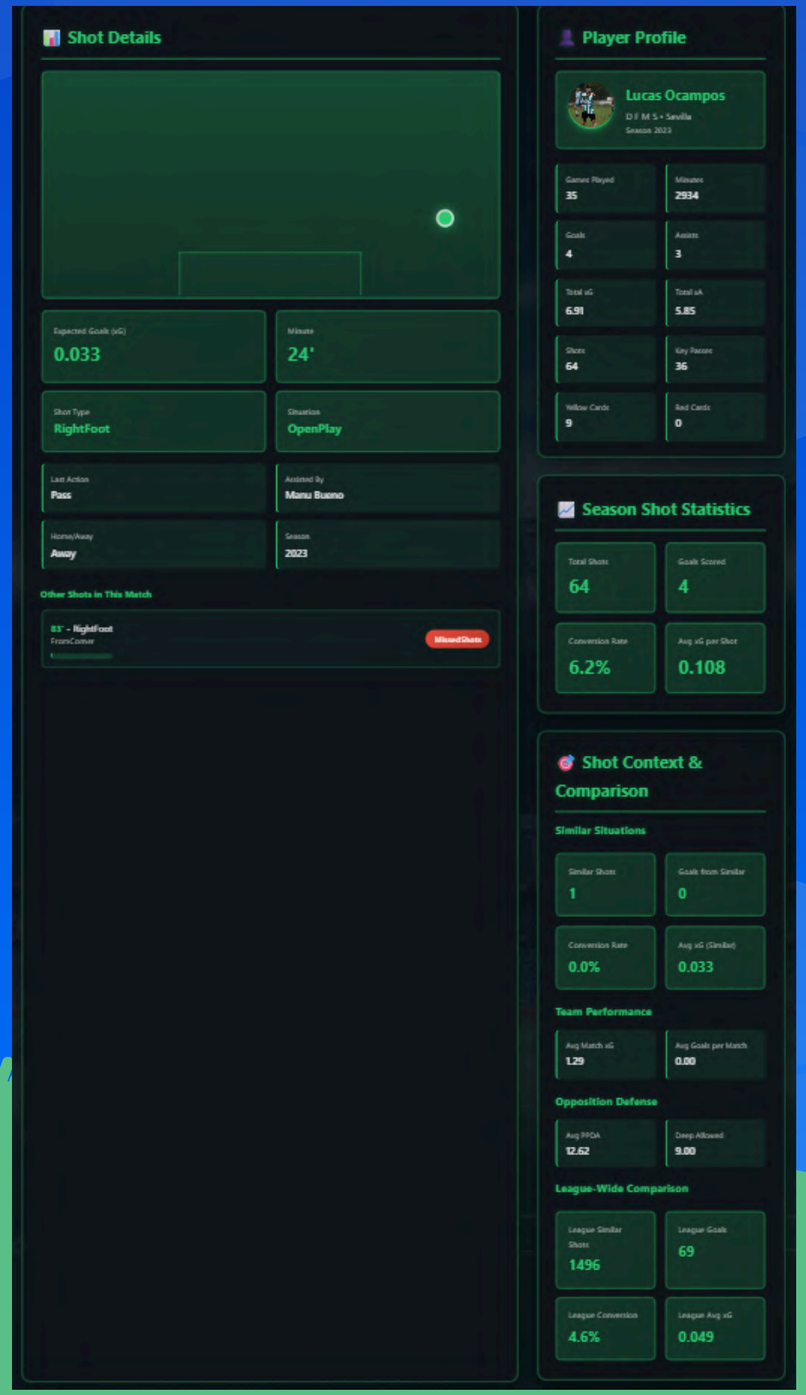
- ADVANCED ANALYSIS



FEATURES

- ADVANCED ANALYSIS





NO SQL INJECTION

```python
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username")
        password = request.form.get("password")

        # Validation
        if not username or not password:
            return render_template("login.html", error="Username and password are required"), 400

        # Safe SQL with parameterized queries
        sql = "SELECT * FROM users WHERE username = %s"
        users = db.execute_query(sql, (username,))

        if not users:
            return render_template("login.html", error="Invalid username or password"), 401

        user = users[0]

        if not check_password_hash(user["password_hash"], password):
            return render_template("login.html", error="Invalid username or password"), 401

        # Set session
        session["user_id"] = user["id"]
        session["username"] = user["username"]

        return redirect("/admin")

    error = request.args.get("error")
    registered = request.args.get("registered")
    return render_template("login.html", error=error, registered=registered)
```

Dangerous Code !!

```python
sql = f"SELECT * FROM users WHERE username = '{username}', '{password}'"
users = db.execute_query(sql)
```

We used parameter binding (%s) in our all sql queries, which forces the database to treat user input strictly as data, never as executable code.

TESTING

```python
def verify_foreign_keys():
    """Verify that foreign key constraints are properly set up"""
    conn = connect_db(True)
    cur = conn.cursor()

    query = """
        SELECT
            TABLE_NAME,
            COLUMN_NAME,
            CONSTRAINT_NAME,
            REFERENCED_TABLE_NAME,
            REFERENCED_COLUMN_NAME
        FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
        WHERE TABLE_SCHEMA = %s
        AND REFERENCED_TABLE_NAME IS NOT NULL
        ORDER BY TABLE_NAME;
    """

    cur.execute(query, (DB_NAME,))
    results = cur.fetchall()

    print("\n🔗 Foreign Key Constraints:")
    if results:
        for row in results:
            print(f"   {row[0]}.{row[1]} -> {row[3]}.{row[4]} (constraint: {row[2]})")
    else:
        print("   ⚠️ No foreign key constraints found!")

    cur.close()
    conn.close()
```

```
🔗 Foreign Key Constraints:
   fut23.team_id -> teams.team_id (constraint: fut23_ibfk_1)
   player.best_shot_id -> shot_data.shot_id (constraint: player_ibfk_1)
   season.team_id -> teams.team_id (constraint: season_ibfk_1)
   shot_data.match_id -> match_info.match_id (constraint: shot_data_ibfk_1)
```

ANY QUESTIONS?